UNIVERSIDAD
TORCUATO DI TELLA

Escuela de Negocios
Licenciatura en Tecnología Digital

Optimización de Packing de cajas en Contenedores

TD8: Informe Final

Autores: Catalina Sofía Dolhare, Joaquin Schanz
Profesor: Pablo Nussembaum
Buenos Aires, 30 de noviembre de 2025

Índice

1. Agradecimientos.... 4
2. Resumen Ejecutivo.... 4
3. Introducción.... 5
4. Justificación del tema.... 5
5. Objetivos de Proyecto.... 6
    5.1 Objetivo general.... 6
    5.2 Objetivos específicos.... 7
6. Planteamiento del problema.... 8
7. Hipótesis.... 9
8. Marco teórico.... 10
    8.1 Programacion Lineal Entera.... 10
    8.2 Problemas de empaquetamiento.... 11
    8.3 NP-Completitud.... 11
    8.4 Heurísticas.... 12
    8.5 Tecnologías utilizadas.... 13
        8.5.1 CPLEX.... 13
        8.5.2 Java.... 13
9. Marco metodológico.... 14
10. Heurística 1: Relajación Lineal.... 14
    10.1 Algoritmo.... 14
    10.2 Resultados.... 16
    10.3 Ventajas y Limitaciones.... 21
11. Heurística 2: División por capas.... 22
    11.1 Algoritmo.... 22
    11.2 Resultados.... 24
    11.3 Ventajas y Limitaciones.... 27
12. Heurística 3: División por capas con relleno.... 28
    12.1 Algoritmo.... 28
    12.2 Resultados.... 30
    12.3 Ventajas y Limitaciones.... 35
13. Heurística 4: Agrupando cajas.... 36
    13.1 Algoritmo.... 36
    13.2 Resultados.... 39
    13.3 Ventajas y Limitaciones.... 48
14. Discusión.... 49

14.1 Relación entre los resultados obtenidos y el marco teórico.....50
14.2 Comportamiento comparado del modelo exacto y las heurísticas.....50
14.3 Sensibilidad geométrica y efectos del contenedor.....51
14.4 Ventajas, limitaciones e implicancias prácticas del enfoque aplicado.....52
15. Conclusiones & Recomendaciones.....52
16. Referencias bibliográficas.....55
17. Apéndices.....56

1. Agradecimientos

A la Universidad Torcuato Di Tella, por brindarnos la formación académica y los recursos necesarios para el desarrollo de este trabajo. A Javier Marenco y Pablo Nussembaum, por su guía, dedicación y valiosos aportes a lo largo de este proceso de investigación. Finalmente a nuestras familias y amigos, por su apoyo incondicional y aliento constante durante esta etapa.

2. Resumen Ejecutivo

El presente trabajo aborda el problema de empaquetamiento tridimensional de cajas idénticas dentro de contenedores, una tarea de alta relevancia operativa y computacional. A partir de la formulación exacta provista por el proponente[1], basada en un modelo de programación entera que incorpora restricciones geométricas y de estabilidad, se buscó evaluar sus limitaciones prácticas (particularmente el crecimiento exponencial de variables y la consecuente demanda de memoria y tiempo) y desarrollar alternativas capaces de resolver instancias que el modelo base no logra abordar.

El estudio se orientó a comparar sistemáticamente el desempeño del modelo exacto frente a cuatro heurísticas: (1) relajación lineal con filtrado adaptativo de variables; (2) descomposición por capas horizontales; (3) descomposición por capas con llenado tridimensional posterior; y (4) agrupamiento de cajas en bloques. Para ello se definieron criterios homogéneos de evaluación, se ejecutaron todas las instancias correspondientes a los dos primeros contenedores del set original y se registraron tiempos y calidades de solución bajo modalidades automáticas (script en Python) y manuales (Eclipse), según la naturaleza de cada implementación.

Los resultados muestran que el modelo base permite caracterizar el comportamiento del problema, pero enfrenta cuellos de botella significativos en instancias medianas y grandes. En contraste, las heurísticas permiten obtener soluciones factibles en tiempos considerablemente menores, logrando aceleraciones que en algunos casos superan los 5.000× o incluso 10.000× respecto del modelo exacto. Si bien estas ganancias de velocidad pueden implicar pérdidas en la calidad de las soluciones, en ciertos escenarios las heurísticas no solo reproducen los resultados del modelo base, sino que también logran mejorar la cantidad de cajas empacadas.

En conjunto, el estudio demuestra que la incorporación de enfoques heurísticos es esencial para escalar el problema más allá de los límites del método exacto y permite delinear caminos futuros para mejorar la calidad, robustez y generalización de estas estrategias.

[1] Repositorio GitLab del proyecto: https://gitlab.com/jmarenco/3dpacking

3. Introducción

El problema de **3D Packing** surge en numerosos contextos logísticos e industriales, como la organización de carga en contenedores, la planificación de almacenes o el diseño de embalajes. Consiste en ubicar un conjunto de cajas de un tamaño determinado dentro de un contenedor de dimensiones fijas, maximizando el número de elementos colocados sin que se superpongan y respetando restricciones de estabilidad.

En este trabajo de investigación parte de una **formulación de programación entera** que modela el problema de manera exacta, permitiendo decidir la posición y orientación de cada caja dentro del espacio discretizado del contenedor. El modelo incluye condiciones para evitar solapamientos y para garantizar que cada caja tenga, al menos, un 70 % de su base apoyada, aspecto crucial para la seguridad y la factibilidad práctica. A partir de esta base, se analizan sus ventajas y limitaciones y se desarrollan variantes y heurísticas orientadas a obtener soluciones de buena calidad utilizando menos recursos computacionales.

La importancia de este estudio radica en que el empaquetamiento 3D es un problema **combinatorio** y **computacionalmente complejo**, lo que dificulta resolver instancias de tamaño real con métodos exactos. Esto obliga a buscar enfoques que reduzcan el esfuerzo computacional sin perder calidad en las soluciones. En este contexto, el trabajo estudia cómo distintos criterios de discretización, preprocesamiento y construcción heurística permiten evitar los cuellos de botella de memoria y tiempo que presenta el modelo original, y habilitan la resolución de instancias que de otro modo resultarían intratables.

A lo largo del informe se presenta el planteamiento del problema, la metodología aplicada y las mejoras propuestas. Además, se muestran los resultados obtenidos al comparar múltiples heurísticas con el método base y entre ellas, destacando en qué contextos se logra reducir drásticamente los tiempos de resolución, en cuáles se alcanza una calidad similar y en qué casos incluso se supera la cantidad de cajas empacadas. Estos hallazgos permiten anticipar que, aun sin sacrificar significativamente la calidad de las soluciones, es posible escalar a instancias que el modelo original no logra resolver dentro de los límites operativos habituales.

4. Justificación del tema

El empaquetamiento tridimensional de cajas en contenedores es un problema de gran relevancia tanto académica como práctica. En el ámbito científico, representa un desafío clásico de optimización combinatoria, donde convergen la programación lineal entera, la geometría computacional y la búsqueda de heurísticas avanzadas. Abordarlo permite profundizar en técnicas de modelado matemático y resolución exacta, así como en el desarrollo de algoritmos que equilibran precisión y eficiencia, contribuyendo al avance del campo de la investigación operativa.

Además, el 3D Packing se inscribe dentro de una familia de problemas (los NP-Completos) para los que no se conocen algoritmos polinomiales que garanticen soluciones óptimas para todas las instancias. Esta característica lo convierte en un caso de estudio sumamente desafiante dentro de la optimización combinatoria; ya que incluso pequeñas modificaciones en las dimensiones, en las orientaciones permitidas o en la discretización del espacio pueden cambiar drásticamente la dificultad del modelo. Trabajar con un problema de estas características resulta estimulante a nivel mental, porque invita a explorar estrategias que van desde formulaciones de programación entera refinadas hasta abordajes heurísticos capaces de producir soluciones de calidad en tiempos razonables.

Desde una perspectiva aplicada, la importancia del tema es evidente: mejorar la utilización del espacio en los contenedores significa reducir costos logísticos, disminuir la huella de carbono y acelerar la distribución de bienes, incluyendo insumos críticos como alimentos o medicamentos. En este último caso, las restricciones de estabilidad presentes en el modelo cobran un sentido mayor, ya que muchos medicamentos necesitan ser transportados en condiciones estrictas (sin vuelcos, golpes o roturas) para mantener su usabilidad y eficacia, y suelen tener un alto costo unitario. Cada incremento en la eficiencia de empaquetamiento puede traducirse en menos envíos necesarios y, por ende, en un impacto ambiental, sanitario y económico positivo.

El proyecto resulta especialmente motivador porque combina un reto intelectual exigente con un propósito concreto y socialmente valioso. Nos brinda la oportunidad de aplicar los conocimientos adquiridos en programación lineal entera y optimización exacta; y funciona como laboratorio para poner a prueba ideas, comparar enfoques y comprender más profundamente los límites computacionales de la optimización moderna. De este modo, el trabajo no solo contribuye al desarrollo académico en optimización, sino que también responde a necesidades reales de la industria y la sociedad.

5. Objetivos de Proyecto

5.1 Objetivo general

Analizar en profundidad el problema de 3D Packing y comprender cómo las limitaciones de recursos computacionales se manifiestan en el modelo base recibido, identificando su impacto en distintas instancias del problema. A partir de este entendimiento, desarrollar y evaluar una estrategia de optimización que permita resolver el problema de empaquetamiento tridimensional de cajas en contenedores, incorporando técnicas heurísticas que permitan superar las restricciones y limitaciones de memoria presentes en el modelo original, y, de esta manera, avanzar hacia soluciones más eficientes y escalables en contextos puntuales. De ser posible, aventurarse a señalar tendencias y posibles generalizaciones en base a los resultados observados en instancias puntuales.

5.2 Objetivos específicos

1. Establecer un conjunto de parámetros y criterios de evaluación en los que posteriormente se apoyará la comparación entre las soluciones. Esto incluye definir métricas de desempeño, indicadores de calidad y medidas vinculadas al uso de recursos computacionales, de modo que todas las alternativas puedan analizarse bajo estándares homogéneos y transparentes.
2. Evaluar sistemáticamente el desempeño del modelo base en cada una de las instancias provistas por el proponente, prestando especial atención a cómo fluctúan el tiempo de resolución y la calidad de las soluciones construidas en función de las dimensiones específicas del contenedor y las cajas. Para así poder identificar patrones de comportamiento, puntos críticos y posibles factores que condicionan la eficiencia del modelo.
3. A partir de la evidencia recolectada, idear planteamientos heurísticos que aprovechen las particularidades observadas en las instancias que se resuelven de manera más eficiente y con mejores resultados, reconociendo que en resoluciones de estas características existe un trade-off ineludible entre la calidad de la solución y el tiempo de respuesta, y utilizar dicho conocimiento para orientar la construcción de heurísticas adaptadas a las restricciones y características del problema.
4. Implementar estas propuestas heurísticas en el entorno de trabajo sugerido por el proponente, asegurando una integración adecuada con el código existente y verificando que cada implementación se comporte efectivamente según lo esperado desde el punto de vista lógico, matemático y computacional.
5. Evaluar rigurosamente cada uno de los planteamientos desarrollados sobre las mismas instancias utilizadas en el análisis del modelo base, registrando de forma detallada los resultados obtenidos con respecto a los parámetros establecidos. Para así identificar mejoras, retrocesos o comportamientos diferenciados en función de las características específicas de cada instancia.
6. Comparar los resultados derivados de las diferentes ideas tanto de manera interna como cruzada. Es decir, analizar cómo se comportan las distintas formulaciones (exactas y heurísticas) frente a una misma instancia, y cómo varía el desempeño de una misma formulación frente a distintas instancias. Esto en pos de explicar qué propiedades estructurales del problema favorecen o perjudican el rendimiento de cada enfoque, y en qué escenarios cada idea resulta particularmente ventajosa.
7. Presentar todos los resultados de forma clara, ordenada y visualmente atractiva, de manera que puedan ser comprendidos, replicados y utilizados por cualquier persona interesada en la temática. Esto implica elaborar tablas, gráficos y descripciones que comuniquen con precisión los hallazgos y faciliten su interpretación.

6. Planteamiento del problema

El problema de empaquetamiento tridimensional (3D bin packing) consiste en determinar la cantidad máxima de cajas idénticas que pueden ser colocadas dentro de un contenedor, respetando restricciones geométricas y de estabilidad física. Este problema, clasificado como NP-hard, tiene aplicaciones significativas en logística, diseño de almacenes, transporte de carga y optimización de espacios.

En el contexto de esta investigación, se aborda una variante del problema que incorpora restricciones de estabilidad realistas junto con restricciones geométricas fundamentales. El problema se define a partir de las dimensiones del contenedor (ancho W, largo L y altura H) y las dimensiones de cada caja idéntica (ancho w, largo l y altura h). El espacio del contenedor se discretiza considerando únicamente posiciones válidas donde pueden ubicarse los vértices inferiores izquierdos de las cajas, generando conjuntos finitos I, J y K para cada dimensión espacial.

El modelo matemático incorpora tres tipos fundamentales de restricciones:

1. **Restricciones de no solapamiento:** Garantizan que ningún par de cajas ocupe simultáneamente el mismo punto del espacio contenedor.
2. **Restricciones de contención:** Aseguran que cada caja colocada en el contenedor esté completamente contenida dentro de sus límites.
3. **Restricciones de estabilidad:** Cada caja debe tener al menos el 70% de la superficie de su base inferior en contacto con otras cajas subyacentes o con el piso del contenedor. Esta restricción refleja condiciones prácticas de apilamiento que garantizan la estabilidad física de la configuración, evitando colapsos.

Adicionalmente, el modelo permite seis orientaciones posibles para cada caja, lo que amplía significativamente el espacio de búsqueda y la flexibilidad de las soluciones, pero también aumenta la complejidad computacional del problema.

Se nos ha proporcionado una implementación completa del modelo matemático utilizando CPLEX como solver y Java como lenguaje de programación. Sin embargo, el modelo exacto presenta limitaciones computacionales importantes que motivan esta investigación. La discretización del espacio del contenedor y la necesidad de considerar múltiples orientaciones posibles para cada caja generan un número exponencial de variables binarias x_{ijk}^{p} y restricciones.

Para instancias de tamaño considerable, el modelo enfrenta dos problemas críticos para ciertas instancias. Por un lado, la cantidad de variables y restricciones crece de manera significativa, superando la capacidad de memoria disponible incluso en computadoras modernas. Por otro lado, los tiempos de ejecución para encontrar soluciones óptimas o cercanas al óptimo se vuelven muy altos para aplicaciones reales.

Los resultados experimentales de la implementación exacta, presentados en el Apéndice A, documentan estas limitaciones y establecen una línea base de comparación para las soluciones heurísticas propuestas.

Esta situación vuelve imprescindible explorar métodos heurísticos que, aun sin garantizar optimalidad, permitan encontrar buenas soluciones en tiempos razonables. Estos enfoques no solo abren la posibilidad de resolver instancias que los métodos exactos no pueden abordar, sino que también ayudan a comprender mejor la estructura combinatoria del problema y a reconocer qué estrategias resultan más eficaces para descomponerlo y enfrentarlo de manera iterativa en escenarios de gran escala.

7. Hipótesis

Con base en el planteamiento del problema de empaquetamiento 3D con restricciones de estabilidad y las limitaciones identificadas en el modelo exacto, se propone el desarrollo e implementación de cuatro algoritmos heurísticos alternativos que permitirán abordar instancias de mayor tamaño de manera eficiente.

La implementación de algoritmos heurísticos basados en relajación lineal, descomposición por capas, llenado progresivo y agrupamiento de cajas en bloques permitirá resolver instancias grandes del problema de empaquetamiento 3D con restricciones de estabilidad en tiempos computacionalmente viables, superando las limitaciones de memoria y tiempo del modelo exacto.

Heurística 1: Relajación lineal con selección adaptativa de posiciones

Esta heurística utiliza la relajación lineal del problema como mecanismo de preprocesamiento. Primero resuelve el modelo permitiendo variables continuas en [0,1], lo que en términos reales no tiene sentido ya que se entendería como poner un porcentaje de una caja con cierta orientación en una posición del contenedor (Ejemplo: “colocar el 37% de una caja” o “superponer el 82% de una orientación” en un punto específico del contenedor), pero las cajas son objetos indivisibles y discretos, y su ubicación no admite gradaciones parciales. Luego selecciona un subconjunto de posiciones candidatas basándose en los valores más prometedores (cercanos a 1), y resuelve el modelo entero sobre este subconjunto reducido. Se hipotetiza que este enfoque reducirá significativamente los requerimientos de memoria al sintetizar altamente la cantidad de variables a explorar; al mismo tiempo que se explora eficientemente el espacio de soluciones.

Heurística 2: Descomposición por capas horizontales

Esta heurística descompone el problema tridimensional en subproblemas bidimensionales más tratables, construyendo la solución capa por capa desde el piso hacia arriba. Cada capa se formula como un problema de empaquetamiento 2D independiente que maximiza el número de cajas en ese nivel. Las restricciones de estabilidad se simplifican ya que, al tener todas capas iguales, todas las cajas colocadas

tienen el 100% de superficie cubierta. Se hipotetiza que esta descomposición reducirá drásticamente la complejidad al transformar un problema 3D en múltiples problemas 2D secuenciales.

Heurística 3: Descomposición por capas con llenado tridimensional

Esta heurística extiende la Heurística 2 incorporando una fase adicional de optimización. Tras obtener una solución inicial mediante construcción por capas, identifica espacios residuales en el interior del contenedor y formula un problema de llenado 3D para colocar cajas adicionales en estos espacios, considerando todas las orientaciones y verificando restricciones completas de no solapamiento y estabilidad respecto a todas las cajas ya colocadas. Se hipotetiza que esta heurística combinará la eficiencia de la construcción por capas con la capacidad de aprovechar oportunidades de empaquetamiento tridimensional más complejo, logrando un mejor balance entre calidad y tiempo computacional.

Heurística 4: Agrupamiento de cajas en bloques

Esta heurística se basa en la observación de que cajas de dimensiones mayores, más similares al contenedor reducen el tiempo de resolución al disminuir las combinaciones posibles. El algoritmo preagrupa cajas en bloques de mayor volumen (representados por tuplas {a,b,c} que indican cantidad de cajas en cada dimensión) y resuelve iterativamente el posicionamiento de estos bloques. Se define una secuencia de tipos de bloque, y en cada iteración se resuelve un modelo de optimización independiente que coloca bloques del tipo actual, verificando no solapamiento y estabilidad. Los bloques colocados se registran y se actualiza una máscara de ocupación del contenedor para las iteraciones siguientes. Se hipotetiza que trabajar con unidades de mayor volumen simplificará el problema y acelerará la resolución al rellenar el contenedor más rápido, permitiendo además explorar diferentes estrategias de agrupamiento adaptadas a las proporciones del contenedor. Además, se presupone que la configuración de los bloques seleccionados será significativa en el desempeño de la heurística para cada instancia individual, debido a que no todas las instancias manejan los mismos tamaños, habrá estrategias que resultarán más o menos beneficiosas.

8. Marco teórico

8.1 Programacion Lineal Entera

La programación lineal entera (PLE) es una técnica de optimización matemática mediante la que se busca maximizar o minimizar una función objetivo lineal sujeta a restricciones lineales, con la particularidad de que las variables de decisión, las que hacen a la solución del problema, deben tomar valores enteros. Formalmente, un problema de este tipo se expresa como:

$$maximizar\ c^T x$$

$$sujeto\ a\ Ax \leq b,\ x \in Z^k$$

donde c es el vector de coeficientes de la función objetivo, A es la matriz de coeficientes de las restricciones, b es el vector de términos independientes, y x es el vector de variables enteras. (Wolsey, 1998). Un caso particularmente relevante es la programación binaria, donde las variables están restringidas al conjunto {0,1}, que es precisamente el enfoque utilizado en el problema de empaquetamiento 3D abordado en este trabajo.

8.2 Problemas de empaquetamiento

Los problemas de empaquetamiento constituyen una clase de problemas de optimización combinatoria con amplias aplicaciones. El caso de los problema de empaquetamiento tridimensional (3D Bin Packing Problem), como es el de este problema, busca ubicar objetos tridimensionales dentro de uno o más contenedores, optimizando algún tipo de criterio, como por ejemplo, el número de objetos empaquetados (Wäscher et al., 2007).

En aplicaciones prácticas aparecen restricciones adicionales como estabilidad, restricciones de orientación, y restricciones de carga. Estas restricciones reflejan consideraciones físicas y operacionales que aumentan la complejidad del problema (Bortfeldt & Wäscher, 2013).

Para estos problemas resueltos con modelos de programación entera, es necesario discretizar el espacio continuo del contenedor. La discretización garantiza que si existe una solución óptima al problema continuo, entonces existe una solución óptima donde todos los objetos tienen vértices en puntos de la discretización, preservando así la optimalidad (Scheithauer, 1999).

8.3 NP-Completitud

La teoría de la complejidad computacional clasifica los problemas según los recursos necesarios para resolverlos. La clase P contiene problemas resolubles en tiempo polinomial, mientras que la clase NP contiene problemas cuyas soluciones pueden verificarse en tiempo polinomial (Garey & Johnson, 1979). En particular, $P \subseteq NP$ ya que un problema que la solución de un problema que se resuelva en tiempo polinomial, puede verificarse en tiempo polinomial.

Un problema es NP-completo si se cumplen las siguientes dos condiciones: pertenece a NP y cualquier otro problema en NP puede reducirse a él en tiempo polinomial. Estos problemas NP-completos son los más complejos dentro de la clase NP. Por otro lado, si sólo se verificará la segunda condición

mencionada, se considera que el problema es NP-difícil (NP-hard), estos son al menos tan difíciles como los problemas NP-completos.

Una de las cuestiones abiertas más importantes en ciencias de la computación es si P = NP. Esta pregunta interroga si todos los problemas cuyas soluciones pueden verificarse eficientemente también pueden resolverse eficientemente. Si P = NP, entonces existirían algoritmos polinomiales para todos los problemas NP-completos; si P ≠ NP, tales algoritmos no existen (Garey & Johnson, 1979).

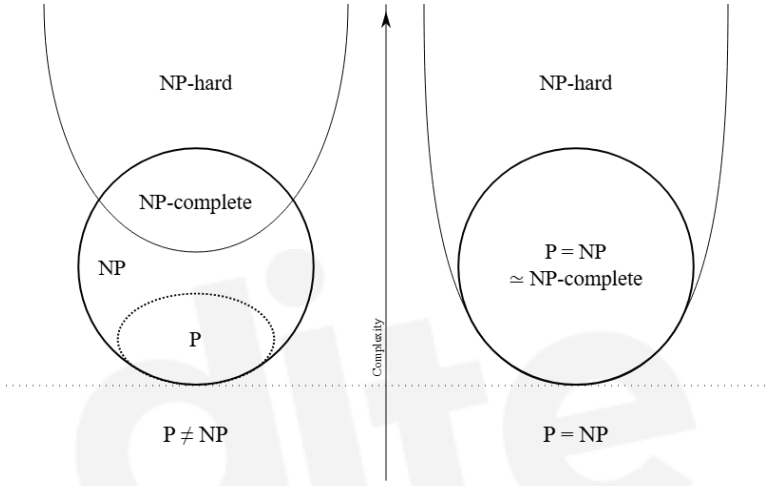

Figura 1: Diagrama de las clases de complejidad computacional bajo los escenarios P ≠ NP y P = NP. Fuente: Wikimedia Commons (2024).

El problema de empaquetamiento tridimensional es NP-difícil, demostrado mediante reducción desde problemas conocidos como el 3-Partition (Garey & Johnson, 1979). Esto implica que no se conocen algoritmos que garanticen encontrar la solución óptima en tiempo polinomial, y bajo la conjetura P ≠ NP, tales algoritmos no pueden existir. Los algoritmos exactos disponibles tienen complejidad exponencial en el peor caso, lo que los hace inviables para instancias de tamaño grande. Esta característica del problema motiva el desarrollo de enfoques heurísticos que sacrifican la garantía de optimalidad a cambio de tiempos de ejecución razonables.

8.4 Heurísticas

Las heurísticas son estrategias de resolución que buscan encontrar soluciones de buena calidad para problemas computacionalmente difíciles. A diferencia de los métodos exactos, no exploran exhaustivamente todo el espacio de soluciones ni dan garantía de encontrar el óptimo global; por este motivo se pierde la exactitud. Una solución heurística puede ser factible y buena en términos prácticos,

pero no existe un mecanismo interno que permita certificar si es realmente la mejor solución posible o cuán lejos está de la mejor solución posible.

Aun así, su utilidad radica en que generan soluciones que sí cumplen las restricciones del modelo, por lo que siempre proporcionan configuraciones factibles del problema. Dependiendo del enfoque, pueden construirse soluciones desde cero (heurísticas constructivas), mejorar soluciones ya existentes (heurísticas de mejoramiento) o valerse de mecanismos más amplios de exploración, como en el caso de las metaheurísticas (Talbi, 2009). En conjunto, estos métodos permiten obtener respuestas viables en tiempos considerablemente menores que los requeridos por técnicas exactas, aunque sin la garantía de optimalidad.

Este trade-off entre optimalidad garantizada y eficiencia computacional hace que las heurísticas sean especialmente valiosas para problemas NP-difíciles de gran escala, donde los métodos exactos pueden requerir tiempos de ejecución impracticables.

8.5 Tecnologías utilizadas

8.5.1 CPLEX

IBM ILOG CPLEX es uno de los solvers de optimización más avanzados y utilizados para resolver problemas de programación lineal, programación entera y programación cuadrática. Este fue desarrollado por IBM e implementa distintos algoritmos como el método simplex dual y primal, métodos de punto interior, y técnicas sofisticadas de branch-and-cut para problemas enteros (IBM, 2021).

A traves de una API, CPLEX permite construir modelos de optimización en diversos lenguajes de programación, como Java, C++, Python y C#. La API proporciona clases y métodos para definir variables, restricciones y funciones objetivo, así como para configurar parámetros del solver y extraer resultados (IBM, 2021).

8.5.2 Java

Java es un lenguaje de programación orientado a objetos de propósito general desarrollado por Sun Microsystems (ahora Oracle) en 1995. Java se caracteriza por su portabilidad multiplataforma gracias a la Java Virtual Machine (JVM), que permite ejecutar el mismo código compilado en diferentes sistemas operativos (Gosling et al., 2014). El paradigma orientado a objetos (POO) de Java organiza el software en clases, facilitando la reutilización y mantenibilidad del código.

Para este proyecto, Java proporciona las estructuras necesarias para implementar el modelo de empaquetamiento, gestionar matrices multidimensionales de variables y objetos, e interactuar eficientemente con la API de CPLEX.

9. Marco metodológico

El enfoque metodológico del proyecto se basó en un proceso de ideación, experimentación y comparación de resultados, orientado a evaluar el desempeño de distintas heurísticas aplicadas al problema. Para ello, dentro del conjunto total de instancias disponibles compartido por el proponente, se optó por trabajar únicamente con los dos primeros contenedores y todas las instancias correspondientes, eliminando aquellas que se encontraban repetidas, ya que se consideró que eran suficientes y representativas para realizar las pruebas y comparar los resultados.

En primer lugar, se ejecutó el modelo base para todas las instancias seleccionadas con el fin de establecer una línea de referencia común. Luego se evaluaron las heurísticas previamente definidas, utilizando dos modalidades de ejecución según correspondiera; algunas de ellas se corrieron mediante un script en Python que automatiza la ejecución del programa principal implementado en Java, mientras que otras se ejecutaron directamente desde Eclipse. En todos los casos se registraron de forma homogénea los resultados relevantes (la cantidad de cajas ubicadas y el tiempo de ejecución) junto con las métricas adicionales específicas de cada enfoque cuando correspondía.

Este esquema combinó automatización y ejecución manual según lo requería cada implementación, manteniendo la coherencia de los datos recolectados y permitiendo comparar cada heurística contra la línea de base establecida.

10. Heurística 1: Relajación Lineal

10.1 Algoritmo

La heurística de relajación lineal implementa una estrategia de preprocesamiento en dos fases para reducir el tamaño del modelo de programación entera antes de resolverlo. El objetivo es eliminar variables que tienen baja probabilidad de formar parte de la solución óptima, reduciendo así el espacio de búsqueda y mejorando el tiempo de resolución.

Fase 1: Relajación lineal

En esta primera fase se resuelve una versión relajada del modelo. En este caso, se mantiene exactamente la misma estructura que el modelo original en términos de función objetivo y restricciones, con una diferencia fundamental: la forma en que se crean las variables.

La diferencia clave respecto al modelo original radica en el método `createVariablesRelaxed()`, que reemplaza al método estándar createVariables(). En lugar de crear variables binarias mediante `_cplex.boolVar()`, este nuevo método crea variables continuas utilizando `_cplex.numVar(0.0,`

`1.0)`. Esto significa que cada variable `x[i][j][k][p]` puede tomar cualquier valor real en el intervalo [0,1], en lugar de estar restringida únicamente a los valores 0 o 1. Esta modificación transforma el problema de programación entera en un problema de programación lineal, que es significativamente más rápido de resolver.

Una vez creadas las variables continuas, se procede a construir el resto del modelo exactamente como en la versión original y se resuelve mediante CPLEX. Como resultado, a cada variable se le asigna un valor real entre 0 y 1. Variables con valores cercanos a 1 indican posiciones que probablemente serán útiles en la solución entera, mientras que variables con valores cercanos a 0 sugieren posiciones que difícilmente formarán parte de la solución óptima.

Fase 2: Filtrado de variables

Para el filtrado de variables, se define un parámetro umbral ε (epsilon) con el cual experimentamos con 4 valores posibles: $\varepsilon = \{0.0001,\ 0.001,\ 0.01,\ 0.1\}$. Para cada variable que obtuvo un valor menor que ε en la solución relajada, se procede a eliminarla del modelo restringido a variables enteras. La eliminación se realiza mediante el método `removeVariable()`.

La idea detrás de este filtrado es que si una variable tiene un valor muy pequeño en la relajación lineal, donde tiene mayor libertad para tomar valores, entonces es altamente improbable que esta variable tome el valor 1 en la solución entera óptima, donde las restricciones son más estrictas.

Fase 3: Resolución del Modelo Entero Reducido

Una vez completado el filtrado, se inicia la tercera fase donde se resuelve el modelo de programación entera con el conjunto reducido de variables.

La creación de variables en esta segunda fase utiliza el método estándar `createVariables()`, que genera variables binarias. Sin embargo, existe una diferencia crucial con respecto al modelo original: el método recorre todas las posiciones y orientaciones, pero solo crea variables binarias para aquellas posiciones donde la variable correspondiente no fue eliminada en la Fase 2. Esta verificación se realiza comprobando si `_x[i][j][k][p] != null`, lo que indica que la variable sobrevivió al proceso de filtrado. De esta manera, se construye un modelo entero que contiene únicamente las variables que mostraron ser relevantes en la relajación lineal.

Una vez que se tienen las variables correctas, se vuelve a construir el modelo como el original, con la misma función objetivo y restricciones. Este es resuelto por CPLEX una vez más.

10.2 Resultados

10.2.1 Contenedor 260x240x110

Para el contenedor de dimensiones 260x240x110, a simple vista, podemos ver que a medida que se aumenta el valor de epsilon, es decir, aumentamos el valor mínimo que tienen que tener las variables para ser consideradas en la solución entera, aumenta la cantidad de óptimos encontrados. Sin embargo, al analizar detalladamente los resultados, nos encontramos con que no necesariamente el óptimo encontrado es el óptimo para esa instancia, si no que el óptimo para esa instancia con la cantidad de variables que tiene luego de la relajación. Los resultados completos se pueden ver en el **Apéndice B.**

Tabla 1: Resultados agregados para los valores de epsilon de la heurística N° 1 en comparación al modelo base para el contenedor 260x240x110.

Valor epsilon	Base	0,0001	0,001	0,01	0,1
Óptimos	29	29	34	34	34
Factibles	6	4	0	0	0
Timeout	0	2	1	1	1

Para este contenedor, por más que se encuentren 34 soluciones óptimas para valores de ε por encima de 0,001 no significa que todas aumentaron la cantidad de cajas que caben dentro del contenedor. En la tabla 2, podemos ver los únicos dos casos que, a medida que se aumenta el valor de ε, la cantidad total de cajas aumenta (y luego se mantiene igual). En ambos casos, notamos que el aumento de cajas ocurre cuando $\varepsilon = 0.001$, en el cual ambos obtienen una solución óptima luego de un timeout o una solución factible.

Tabla 2: Resultados de instancias en las que aumenta la cantidad de cajas a medida que crece el valor de ε para el contenedor 260x240x110.

	Base	0,0001	0,001	0,01	0,1	Base	0,0001	0,001	0,01	0,1
Caja	**Total cajas**					**Estado**				
(41, 24, 113)	53	53	56	56	56	Feasible	Feasible	Optimal	Optimal	Optimal
(45, 32, 84)	43		48	48	48	Feasible	Timeout	Optimal	Optimal	Optimal

Por otro lado, encontramos casos en los que sucede lo contrario. En la tabla 3, podemos ver casos en los que aumentar tanto el valor de ε disminuye la cantidad de cajas que se colocan en el contenedor. Esto se debe a que se eliminan demasiadas variables que hace que no se encuentren mejores soluciones. Adicionalmente, podemos ver que aunque la cantidad de cajas sea menor, el modelo nos sigue dando que la solución es óptima. Esto es resultado de una eliminación de variables que son importantes para la

solución global. Por más que se logre una solución óptima para esa cantidad de variables, pero necesariamente es óptima para el problema completo.

Tabla 3: Instancias en las que el total de cajas disminuye dando una falsa optimalidad para el contenedor 260x240x110.

	Base	**0,0001**	**0,001**	**0,01**	**0,1**	**Base**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja	**Total cajas**					**Estado**				
(40, 35, 75)	63	63	63	63	56	Optimal	Optimal	Optimal	Optimal	Optimal
(55, 40, 80)	36	36	35	35	35	Optimal	Optimal	Optimal	Optimal	Optimal
(60, 20, 141)	29	29	26	26	24	Optimal	Optimal	Optimal	Optimal	Optimal
(64, 39, 93)	25	25	25	25	24	Optimal	Optimal	Optimal	Optimal	Optimal
(135, 29, 47)	32	32	32	32	29	Feasible	Feasible	Optimal	Optimal	Optimal
(135, 29, 46)	32	32	28	28	28	Feasible	Feasible	Optimal	Optimal	Optimal
(138, 30, 47)	30	30	28	28	28	Optimal	Optimal	Optimal	Optimal	Optimal

En cuanto a los tiempos de resolución, podemos ver en la tabla 4 las 27 instancias en el que a medida que se aumenta el valor de epsilon el total de cajas se mantiene igual pero el tiempo de resolución decrece. Esto implica una mejora en la solución. Entre estas, se pueden destacar instancias como la caja (45, 30, 84) o (135, 29, 47) que a partir de un $\varepsilon = 0,001$ se ven aceleraciones con respecto al tiempo base de aproximadamente 10000x y 9000x respectivamente. Por otro lado, podemos encontrar casos en los que en la primera relajación lineal, con $\varepsilon = 0,0001$, el tiempo de resolución aumenta, pero a medida que se aumenta el valor de ε, se ve como el tiempo decrece en relación al base y entre cada valor de ε. Esto se puede ver en instancias como en la caja (40, 43, 84) en el cual el tiempo de resolución base es de 28,24 segundos y el de la relajación lineal con $\varepsilon = 0,0001$ es de 36,85 segundos, lo cual representa una aceleración de 0,77x (desaceleración), pero en la siguiente relajación, con un $\varepsilon = 0,001$ podemos ver que el tiempo de resolución pasa a 0,07 segundos, representando una aceleración de 385,78x. Esto mismo se puede ver en instancias como (40, 37, 75) o (41, 37, 75) donde en la primera relajación lineal el tiempo de resolución es mayor al caso base y en las siguientes relajaciones el tiempo disminuye drásticamente. Cabe destacar que, más allá de estos patrones, el valor específico de ε no muestra una influencia clara y consistente en el factor de aceleración, lo cual sugiere que el beneficio principal proviene de la eliminación de variables en sí misma.

Tabla 4: Instancias donde el total de cajas se mantiene al incrementar el valor de ε para el contenedor 260x240x110.

		Base	0,0001	0,001	0,01	0,1	0,0001	0,001	0,01	0,1
Caja	**Total cajas**	**Tiempo Resolución**					**Aceleración**			
(30, 30, 85)	72	1,04	0,66	0,05	0,04	0,04	1,58	19,89	23,76	29,67
(30, 30, 84)	72	1,03	0,60	0,05	0,04	0,03	1,72	19,45	22,82	29,45
(40, 43, 84)	36	28,24	36,85	0,07	0,03	0,03	0,77	385,78	918,75	878,48
(40, 40, 75)	36	0,32	0,13	0,06	0,04	0,05	2,48	5,43	8,71	6,78
(40, 45, 84)	36	42,44	52,05	0,05	0,04	0,04	0,82	922,53	985,01	1121,80
(40, 40, 76)	36	0,31	0,12	0,05	0,05	0,04	2,55	5,91	6,39	6,99
(40, 37, 75)	42	253,39	294,13	0,23	0,15	0,10	0,86	1110,89	1703,92	2534,04
(41, 37, 75)	42	396,47	499,22	0,07	0,08	0,07	0,79	5375,20	5087,73	5685,58
(45, 30, 84)	54	603,47	600,61	0,06	0,06	0,07	1,00	10081,42	9364,15	8890,60
(50, 45, 109)	25	2,81	2,52	0,09	0,09	0,05	1,11	30,52	32,00	53,49
(50, 50, 91)	25	0,19	0,03	0,04	0,05	0,04	7,05	4,55	4,16	4,42
(62, 39, 92)	25	6,28	6,78	0,68	0,63	0,07	0,93	9,25	10,03	93,96
(69, 40, 95)	24	3,05	3,11	0,36	0,30	0,06	0,98	8,46	10,28	55,25
(125, 19, 118)	20	1,14	0,25	0,05	0,04	0,03	4,63	23,86	30,94	33,57
(130, 55, 75)	12	0,18	0,02	0,05	0,04	0,04	8,27	3,71	4,58	4,73
(130, 45, 62)	17	0,77	0,35	0,08	0,05	0,05	2,22	10,26	15,02	14,16
(135, 29, 47)	32	604,22	600,33	0,06	0,06	0,06	1,01	9547,12	9809,42	10366,95
(140, 49, 52)	16	0,84	0,41	0,05	0,04	0,04	2,06	15,68	20,47	21,08
(160, 33, 46)	22	4,60	3,62	0,04	0,03	0,03	1,27	120,26	135,14	148,13
(164, 67, 73)	4	0,14	0,01	0,04	0,03	0,03	11,63	3,60	4,55	4,67
(170, 85, 45)	7	0,19	0,02	0,04	0,04	0,03	7,63	5,15	4,42	5,63
(171, 35, 70)	12	0,18	0,03	0,06	0,05	0,06	6,65	3,08	3,34	2,86
(185, 55, 55)	10	0,15	0,01	0,04	0,03	0,03	13,48	4,02	4,51	4,37
(185, 55, 56)	10	0,26	0,03	0,07	0,04	0,08	8,24	3,81	6,41	3,37
(190, 175, 45)	2	0,12	0,01	0,05	0,03	0,03	13,97	2,37	4,04	3,79
(195, 36, 130)	6	0,17	0,01	0,06	0,04	0,04	13,18	2,56	4,19	4,24

10.2.2 Contenedor 400x240x130

Para el contenedor de dimensiones 400x240x130, a simple vista, podemos ver que a medida que se aumenta el valor de epsilon, la cantidad de resultados óptimos se mantiene igual. En este contenedor, notamos que en el modelo base se tienen 22 óptimos, 13 factibles y 3 no se completaron por límite de tiempo. Una vez que incorporamos la relajación lineal, los 10 factibles pasaron a no poder resolverse por límite de tiempo. Los resultados completos se pueden ver en el **Apéndice B.**

Tabla 5: Resultados agregados para los valores de epsilon de la heurística N° 1 en comparación al modelo base para el contenedor 400x240x130

Valor epsilon	Base	0,0001	0,001	0,01	0,1
Óptimos	22	22	22	22	22
Factibles	10	0	0	0	0
Timeout	3	13	13	13	13

En este caso, analizando la cantidad de cajas que se pueden colocar en el contenedor, la cantidad no cambia a medida que se aumenta el valor de ε, manteniendo óptimas las 22 cajas que se encuentran en el caso base. Sin embargo, aunque el modelo diga que las soluciones son óptimas, ocurre lo mismo que en el contenedor anterior, se disminuye la cantidad de cajas y aun así son soluciones óptimas. En la tabla 7, se pueden ver los casos en los que el valor de las cajas es menor al de el base para algún valor de ε pero aun así sigue siendo óptimo. Esto es un resultado de la eliminación de variables durante el preprocesamiento de la relajación lineal, donde se podría llegar a eliminar ciertas variables que podrían haber contribuido a una solución con mayor cantidad de cajas.

Tabla 6: Instancias en las que el total de cajas disminuye dando una falsa optimalidad para el contenedor 400x240x130.

	Base	0,0001	0,001	0,01	0,1	Base	0,0001	0,001	0,01	0,1
Caja	**Total cajas**					**Estado**				
(30, 30, 85)	149	146	146	146	146	Optimal	Optimal	Optimal	Optimal	Optimal
(30, 30, 84)	149	148	148	148	148	Optimal	Optimal	Optimal	Optimal	Optimal
(40, 40, 75)	90	89	89	89	87	Optimal	Optimal	Optimal	Optimal	Optimal
(40, 40, 76)	90	89	89	89	86	Optimal	Optimal	Optimal	Optimal	Optimal
(50, 45, 109)	40	40	40	40	38	Optimal	Optimal	Optimal	Optimal	Optimal
(60, 20, 141)	60	58	58	58	57	Optimal	Optimal	Optimal	Optimal	Optimal
(170, 85, 45)	15	12	12	12	12	Optimal	Optimal	Optimal	Optimal	Optimal

Las instancias óptimas que restan mantienen la misma cantidad de cajas para cualquier valor de ε. Lo que se destaca de la heurística en estos casos son los tiempos de ejecución, vistos en la tabla 7. Para todos estos casos, los tiempos se reducen significativamente, con factores de aceleración que varían desde aproximadamente 3x hasta más de 8.000x respecto al caso base. Los casos más destacados son las instancias (62, 39, 92) y (64, 39, 93), donde el tiempo de resolución pasa de más de 300 segundos a menos de 0,1 segundos, logrando aceleraciones superiores a 5.000x sin sacrificar la calidad de la solución. Incluso en instancias donde el caso base ya presenta tiempos relativamente bajos, como (50, 50, 91) con 0,285 segundos, la heurística logra reducirlos a menos de 0,05 segundos.

Al igual que en el contenedor anterior, se observa que el valor específico de ε no muestra un patrón claro de influencia en el factor de aceleración; por ejemplo, en la instancia (55, 40, 80) el valor ε = 0,1 produce una aceleración de 677x mientras que ε = 0,0001 logra solo 26x, pero en otras instancias como (62, 39, 92) los factores se mantienen relativamente estables entre 7.000x y 8.000x independientemente del valor de ε. Esto refuerza la conclusión de que el beneficio principal proviene de la eliminación de variables en sí misma, más que del umbral específico empleado.

Tabla 7: Instancias donde el total de cajas se mantiene al incrementar el valor de ε para el contenedor 400x240x130.

		Base	**0,0001**	**0,001**	**0,01**	**0,1**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja	**Total cajas**	**Tiempo resolución**					**Aceleración**			
(50, 50, 91)	**40**	0,285	0,044	0,037	0,046	0,045	6,473	7,623	6,168	6,399
(55, 40, 80)	**63**	43,825	1,635	1,580	1,310	0,065	26,800	27,742	33,462	677,686
(62, 39, 92)	**50**	301,127	0,043	0,038	0,037	0,038	7.074,521	7.983,282	8.160,026	7.874,887
(64, 39, 93)	**50**	328,600	0,066	0,071	0,063	0,057	5.000,533	4.633,788	5.219,568	5.752,449
(69, 40, 95)	**42**	39,617	0,040	0,036	0,036	0,041	978,940	1.099,934	1.115,353	966,056
(125, 19, 118)	**42**	78,761	0,039	0,043	0,038	0,046	2.004,489	1.833,959	2.052,420	1.717,280
(130, 55, 75)	**22**	0,996	0,122	0,109	0,211	0,056	8,135	9,151	4,725	17,722
(130, 45, 62)	**32**	10,533	0,142	0,157	0,124	0,053	73,984	66,930	84,845	200,614
(140, 49, 52)	**26**	5,220	0,051	0,033	0,030	0,031	102,698	156,992	172,515	166,491
(164, 67, 73)	**7**	0,153	0,041	0,044	0,044	0,047	3,704	3,505	3,459	3,249
(171, 35, 70)	**19**	0,243	0,062	0,060	0,059	0,076	3,891	4,045	4,088	3,204
(185, 55, 55)	**18**	0,136	0,054	0,047	0,048	0,045	2,534	2,871	2,816	2,990
(185, 55, 56)	**18**	1,274	0,057	0,053	0,049	0,056	22,390	24,031	25,824	22,810
(190, 175, 45)	**4**	0,154	0,037	0,030	0,033	0,033	4,161	5,075	4,667	4,661
(195, 36, 130)	**13**	0,352	0,038	0,037	0,038	0,034	9,261	9,532	9,369	10,322

10.3 Ventajas y Limitaciones

La principal ventaja de esta heurística es la reducción significativa del espacio de búsqueda. Al eliminar variables basándose en la solución de la relajación lineal, se disminuye el tamaño del modelo entero sin recurrir a criterios arbitrarios o puramente heurísticos. La relajación lineal proporciona información valiosa sobre la importancia relativa de cada variable, ya que variables con valores bajos en la relajación tienen fundamentos matemáticos para ser consideradas poco importantes para la solución.

En términos de eficiencia computacional, para instancias en las que el modelo completo presenta dificultades de memoria o tiempos de resolución, esta heurística ofrece una alternativa al reducir la cantidad de variables, por ende, reduciendo la cantidad de memoria que se necesita y el tiempo que se toma en resolver el modelo. Los resultados confirman este beneficio: se observaron factores de aceleración que varían desde 3x hasta más de 10.000x respecto al caso base, con instancias como (62, 39, 92) y (64, 39, 93) en el contenedor 400x240x130 donde el tiempo de resolución pasó de más de 300 segundos a menos de 0,1 segundos, logrando aceleraciones superiores a 5.000x sin sacrificar la calidad de la solución.

Adicionalmente, la heurística también ofrece flexibilidad mediante el parámetro ε al permitir ajustar el balance entre agresividad del filtrado y conservación de variables útiles. Valores más altos de ε producen modelos más pequeños y tiempos de resolución más rápidos pero aumentan el riesgo de eliminar variables importantes, mientras que valores más bajos son más conservadores y reducen el riesgo de eliminar variables importantes, pero producen modelos más similares en complejidad al original. Sin embargo, los resultados muestran que el valor específico de ε no presenta un patrón claro de influencia ni en la calidad de la solución ni en el factor de aceleración. En algunas instancias, valores más altos de ε producen mayores aceleraciones, mientras que en otras los factores se mantienen estables independientemente del umbral utilizado. Esto sugiere que el beneficio principal de la heurística proviene de la eliminación de variables en sí misma, más que del umbral específico empleado.

La limitación más crítica es el riesgo de eliminar variables que forman parte de la solución óptima. La naturaleza heurística del filtrado significa que se sacrifica la garantía de optimalidad. En casos donde se elimina una variable crítica, el modelo reducido puede producir soluciones subóptimas, y hasta considerarlas óptimas cuando no lo son. Esto se observa claramente en los resultados: en ambos contenedores se identificaron casos donde al aumentar ε se incrementa la cantidad de variables eliminadas y se llega a soluciones con menor cantidad de cajas, pero el modelo indica que la solución es óptima. Esto evidencia que la optimalidad reportada corresponde al espacio de búsqueda reducido y no necesariamente al problema original. Por lo tanto, la elección del umbral de filtrado es crucial: un valor demasiado bajo elimina pocas variables y ofrece poco beneficio computacional, mientras que un valor demasiado alto puede descartar variables útiles y degradar la calidad de la solución. Sin embargo, según los comportamientos observados en los resultados, no existe una manera clara de definir el valor óptimo del umbral, ya que este dependerá del contenedor y las instancias que se estén utilizando.

Es importante destacar que el tiempo de resolución que se mide en la heurística es solo el tiempo que tarda en correr el modelo con variables enteras. Podría ocurrir que correr la heurística entera tenga un tiempo mayor dado que tiene que correr la relajación lineal y luego el modelo con variables enteras.

11. Heurística 2: División por capas

11.1 Algoritmo

La heurística por capas explota la estructura del problema al dividir el contenedor en capas horizontales idénticas a lo largo de la dimensión de altura. La idea fundamental es que si todas las cajas tienen la misma orientación, las capas horizontales del contenedor son independientes entre sí y tienen la misma capacidad. Por lo tanto, basta con resolver el problema de empaquetamiento bidimensional en una sola capa y luego multiplicar el resultado por el número de capas que caben en el contenedor.

Esta heurística transforma el problema tridimensional complejo en un problema bidimensional más simple, reduciendo significativamente la complejidad computacional. En lugar de trabajar con variables `x[i][j][k][p]` que representan posiciones en tres dimensiones con seis orientaciones, se resuelve un problema con variables `x[i][j]` en dos dimensiones con una orientación fija.

Algoritmo principal: LayerHeuristic

El algoritmo principal es implementado en la clase `LayerHeuristic`, en este se evalúa para cada orientación posible, cuantas capas horizontales entran por contenedor y cuantas cajas entran por capa.

La idea principal del algoritmo consiste en iterar sobre las seis orientaciones posibles de las cajas: LW, WL, HW, HL, WH, y LH. Para cada orientación, se crea una caja temporal que permite determinar la altura que tendría una caja con esa configuración. Esta información es luego utilizada para calcular cuántas capas completas entran en el contenedor. El número de capas se obtiene mediante la división entera de la altura total del contenedor (H) y la altura de la caja en la orientación actual.

Una vez determinado el número de capas, el algoritmo crea una instancia del modelo bidimensional `CompleteModel2D`, una adaptación del `CompleteModel` del código base. A este, se le pasa por parámetros la instancia del problema, la discretización del espacio, la orientación fija que se está evaluando. Luego, el modelo bidimensional es resuelto, devolviendo la cantidad de cajas que caben en una sola capa del contenedor. Como todas las capas son idénticas, el número total de cajas que caben en el contenedor es simplemente el producto de la cantidad de cajas que entran en la capa por la cantidad de capas. Al tener capas idénticas, no se viola ninguna de las restricciones planteadas.

Luego de calcular la cantidad total de cajas en cada orientación, se devuelve la opción la cual permite la mayor cantidad de cajas en el contenedor.

Modelo Bidimensional: CompleteModel2D

El modelo bidimensional `CompleteModel2D` resuelve el problema de empaquetamiento en una sola capa horizontal del contenedor. Este es una versión simplificada del modelo tridimensional completo, adaptado para trabajar en dos dimensiones con orientación y altura fijas.

El constructor del modelo recibe cuatro parámetros: la instancia del problema que contiene las dimensiones del contenedor y las cajas, la discretización del espacio que define los puntos válidos I×J×K, la orientación específica que todas las cajas deben tener en esta capa, y el índice de la capa (coordenada k) que se está resolviendo. Estos parámetros quedan almacenados como atributos de la clase para su uso posterior.

Con estos datos, se pasa a la construcción y resolución del modelo de programación entera. Al igual que el `CompleteModel` original, se invoca a los métodos que construyen cada componente del modelo: creación de variables, definición de la función objetivo, y generación de restricciones de independencia.

La diferencia fundamental entre este modelo y el original radica en la creación de variables. En lugar de matrices de cuatro dimensiones `x[i][j][k][p]` y `box[i][j][k][p]`, se crean matrices bidimensionales `x[i][j]` y `box[i][j]`. El índice k (altura) pasa a estar fijo en el valor `_fixedLayer` especificado en el constructor, calculado por `LayerHeuristic` en el cálculo de capas. Para cada posición (i,j) en la discretización, se crea una caja temporal con esos índices, el índice k fijo, y la orientación predeterminada y se verifica si esta caja cabe dentro del contenedor usando el método `fits()`. Si es así, se crea una variable binaria `x[i][j]` y se almacena la caja correspondiente en la matriz.

La función objetivo se construye en `createObjective()` de manera análoga al modelo completo, pero con la dimensionalidad reducida. Las restricciones de independencia se generan en `createIndependenceConstraints()` para garantizar que ningún punto del espacio esté cubierto por más de una caja. Es importante destacar que el modelo bidimensional no incluye restricciones de estabilidad. Esto se debe a que todas las capas son idénticas, por lo que siempre se va a cubrir el 100% de la superficie de las cajas al tener una abajo. Esta simplificación es parte de lo que hace que el modelo bidimensional sea mucho más rápido de resolver.

Adicionalmente, `CompleteModel2D` proporciona dos métodos de acceso que permiten al algoritmo principal recuperar los resultados: `getSolutionCount()` retorna el número de cajas en la solución encontrada, y `getSolveTime()` retorna el tiempo en milisegundos que tomó resolver el modelo.

11.2 Resultados

11.2.1 Contenedor 260x240x110

Para el contenedor de dimensiones 260×240×110, se realizó un análisis comparativo entre la heurística por capas y el método base sobre 36 instancias con diferentes dimensiones de cajas. En la tabla 8, podemos observar el comportamiento de los resultados y los resultados completos de todas las instancias se encuentran detallados en el **Apéndice C**.

Tabla 8: Desempeño de la heurística por capas respecto al método base para el contenedor 260x240x110

Resultado	Cantidad de instancias	Porcentualmente
Aumentó la cantidad de cajas	1	2.9%
La cantidad de cajas se mantuvo	15	42,8%
Disminuyó la cantidad de cajas	19	54,3%

La instancia en la cual la heurística logró aumentar la cantidad de cajas, esta es la de la caja con dimensiones 45, 32, 84. En este caso, la heurística encontró una solución con 45 cajas frente a las 43 del método base, representando una mejora del 4.7%. Notablemente, mientras que el método base solo alcanzó el estado "Feasible" en 611.142 segundos, la heurística encontró una solución superior en apenas 0.88 segundos, logrando una aceleración de 694,5× y mejorando simultáneamente la calidad de la solución.

Este resultado es particularmente relevante, ya que demuestra que la descomposición por capas puede explorar configuraciones alternativas que el método exacto que puede estar limitado por restricciones de tiempo o memoria y no lograr optimizar la solución por completo.

El valor principal de la heurística radica en su capacidad para reproducir soluciones óptimas o factibles con tiempos de ejecución drásticamente menores. En la tabla 9, se puede ver que para los 15 casos en los que la solución se mantuvo igual, 14 de esta instancias eran óptimas y uno solo era factible. En 13 de las 15 instancias, el tiempo de resolución de la heurística se reduce significativamente en comparación con el modelo base. logrando aceleraciones de hasta 330x. Adicionalmente, las 2 instancias en las que los tiempos no mejoran, la diferencia es mínima, menor a 0.05 segundos.

Tabla 9: Instancias en las que el total de cajas se mantiene igual en la heurística por capas y en el modelo base para el contenedor 260x240x110.

Caja	Total de cajas	Estado	Tiempo heurística	Tiempo base	Aceleración
(30,30,85)	72	Optimal	0,29	1,039	3,583
(30,30,84)	72	Optimal	0,29	1,027	3,541
(40,43,84)	36	Optimal	0,68	28,237	41,525
(40,40,75)	36	Optimal	0,27	0,318	1,178
(40,45,84)	36	Optimal	0,65	42,440	65,292
(40,40,76)	36	Optimal	0,25	0,309	1,236
(40,37,75)	42	Optimal	1,21	253,392	209,415
(41,37,75)	42	Optimal	1,2	396,472	330,393
(45,35,61)	60	Feasible	1,95	646,780	331,682
(50,45,109)	25	Optimal	0,36	2,809	7,803
(55,40,80)	36	Optimal	0,28	1,360	4,857
(125,19,118)	20	Optimal	0,34	1,137	3,344
(130,55,75)	12	Optimal	0,19	0,182	0,958
(190,175,45)	2	Optimal	0,16	0,123	0,769
(195,36,130)	6	Optimal	0,16	0,166	1,038

En cuanto a los tiempos de resolución, se observan mejoras significativas, especialmente en instancias donde el modelo original requería más de 600 segundos. En la tabla 10 se muestran las instancias con mayores factores de aceleración:

Tabla 10: Instancias resueltas con la heurística por capas con mayores aceleraciones para el contenedor 260x240x110.

Caja	Total de cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(45,30,84)	48	54	0,47	603,468	1283,974
(135,29,46)	16	32	0,66	604,582	916,033
(135,29,47)	16	32	0,66	604,216	915,479
(41,24,113)	48	53	0,71	619,908	873,110
(45,32,84)	45	43	0,88	611,142	694,480

Estos resultados evidencian un trade-off entre calidad de la solución y velocidad. Las mayores aceleraciones, que superan los 900x, se obtienen en instancias donde la solución contiene significativamente menos cajas que el caso base, como en (135,29,46) y (135,29,47) donde se logra

colocar solo la mitad. Esto se debe al enfoque de la heurística: al restringir el problema a capas horizontales idénticas con una única orientación de caja, se simplifica el espacio de búsqueda y se reduce el problema de tres dimensiones a dos, lo cual permite tiempos de resolución menores. Sin embargo, esta simplificación impide explorar configuraciones tridimensionales más complejas que podrían aprovechar mejor el espacio disponible.

11.2.2 Contenedor 400x240x130

Para el contenedor de dimensiones 400×240×130, se realizó el mismo análisis comparativo que en el contenedor anterior, entre la heurística por capas y el método base. Podemos observar el siguiente comportamiento de los resultados:

Tabla 11: Desempeño de la heurística por capas respecto al método base para el contenedor 400x240x130.

Resultado	Cantidad de instancias	Porcentualmente
Aumentó la cantidad de cajas	4	11.4%
La cantidad de cajas se mantuvo	6	17,2%
Disminuyó la cantidad de cajas	25	71,4%

Las 4 cajas en las que se aumentó la cantidad empacada en el contenedor se pueden ver en la tabla 12. Estos casos muestran que la heurística puede superar al método base en calidad de solución. En 3 de las 4 instancias, el método base no logró encontrar ninguna solución dentro del límite de tiempo, mientras que la heurística encontró soluciones en tiempos menores a 17 segundos. Para la instancia (41,24,113), donde el método base sí encontró una solución factible de 89 cajas, la heurística logró mejorarla en 7 unidades adicionales con una aceleración de 366x.

Tabla 12: Instancias en las que la heurística por capas aumentó la cantidad de cajas con respecto a la base para el contenedor 400x240x130.

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(40,37,75)	90	0	12,3	966,653	78,590
(41,37,75)	90	0	9,78	1.094,801	111,943
(41,24,113)	96	89	1,91	700,830	366,927
(45,35,61)	110	0	16,1	1.473,243	91,506

En este contenedor, se sigue mostrando la capacidad de la heurística en reducir los tiempos de solución. En la tabla 13 se muestran las 6 instancias en las que el valor se mantuvo pero el tiempo de resolución disminuye en gran escala, logrando una aceleración de hasta 680x. Estos casos representan el escenario

ideal para la heurística: se obtiene la misma calidad de solución que el método base pero en una fracción del tiempo.

Tabla 13: Instancias en las que el total de cajas se mantiene igual en la heurística por capas y en el modelo base para el contenedor 400x240x130.

Caja	Total cajas	Tiempo heurística	Tiempo base	Aceleración
(40,35,75)	99	0,91	618,844	680,048
(40,40,75)	90	0,52	12,211	23,483
(40,40,76)	90	0,45	4,112	9,138
(50,45,109)	40	1,01	86,058	85,206
(55,40,80)	63	0,48	43,825	91,302
(125,19,118)	42	1	78,761	78,761

En este contenedor también se presenta el trade-off entre calidad de la solución y velocidad observado anteriormente. En la siguiente tabla se muestran las instancias con mayores factores de aceleración:

Tabla 14:Instancias resueltas con la heurística por capas con mayores aceleraciones para el contenedor 400x240x130.

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(45,30,84)	80	95	1,18	632,769	536,245
(45,30,84)	80	95	1,21	632,769	522,950
(138,30,47)	40	49	1,54	666,952	433,086

Estos resultados refuerzan lo visto en el contenedor anterior. Las aceleraciones, que superan los 400x, corresponden a instancias donde la solución contiene entre un 15% y un 18% menos cajas que el caso base. Por ejemplo, en la instancia (45,30,84) la heurística coloca 80 cajas frente a las 95 del método base, una pérdida de 15 cajas, pero reduce el tiempo de resolución de más de 10 minutos a aproximadamente 1 segundo. De manera similar, en (138,30,47) se pierden 9 cajas pero se logra una aceleración de 433x. A diferencia del contenedor anterior donde se observaron pérdidas de hasta el 50%, en este contenedor las diferencias son menores. No obstante, el patrón se mantiene: las mayores ganancias en tiempo vienen acompañadas de pérdidas en la cantidad de cajas empaquetadas.

11.3 Ventajas y Limitaciones

La principal ventaja de esta heurística es su eficiencia computacional, la cual puede verse en la reducción significativa de los tiempos de resolución. Al reducir el problema tridimensional a seis problemas bidimensionales independientes, se disminuye el número de variables y restricciones. Un problema 3D

con dimensiones |I|×|J|×|K|×6 se reduce a seis problemas 2D de dimensiones |I|×|J|, lo que representa una reducción exponencial en complejidad. Los resultados confirman esto ya que se observaron factores de aceleración de hasta 1283x en el contenedor 260x240x110 y hasta 680x en el contenedor 400x240x130.

Adicionalmente, la heurística demostró poder encontrar soluciones en instancias donde el método base no lo lograba dentro del límite de tiempo. En el contenedor 400x240x130, la heurística encontró soluciones para 3 instancias que el método base no pudo resolver, e incluso en un caso superó la solución factible del método base en 7 cajas. Esto sugiere que, para instancias donde el método exacto presenta dificultades al encontrar una solución, la heurística por capas puede ser no solo más rápida sino también más efectiva.

Sin embargo, la heurística presenta limitaciones importantes. Al forzar que todas las cajas tengan la misma orientación, se pierde la flexibilidad de mezclar orientaciones, lo cual podría permitir una mejor solución. Por otro lado, el espacio residual entre capas, cuando H no es múltiplo exacto de la altura de las cajas, se desperdicia completamente, cuando podrían caber más cajas en otras orientaciones. Esto puede verse reflejado en los casos en los que la heurística encuentra peores soluciones que el caso base. Esto se ve explícitamente en los resultados ya que en el 54.3% de las instancias del contenedor 260x240x110 y en el 71.4% del contenedor 400x240x130, la heurística encontró soluciones con menor cantidad de cajas que el método base, con pérdidas que en algunos casos alcanzaron el 50%.

Los resultados evidencian un trade-off entre calidad de la solución y velocidad. Las mayores aceleraciones se obtienen en instancias donde la degradación de la solución es más significativa, como en (135,29,46) y (135,29,47) donde se logra colocar solo la mitad de las cajas pero con aceleraciones superiores a 900x. Esto se debe a que la simplificación que permite tiempos de resolución rápidos es la misma que impide explorar configuraciones tridimensionales más complejas que podrían aprovechar mejor el espacio disponible.

12. Heurística 3: División por capas con relleno

12.1 Algoritmo

La heurística de división por capas con relleno es una extensión de la heurística por capas básica en el cual se aborda una de sus principales limitaciones: el desperdicio del espacio residual. Cuando la altura del contenedor no es múltiplo exacto de la altura de las cajas en la orientación seleccionada, queda un espacio vacío en la parte superior que la heurística básica no aprovecha. Así mismo, si una de las capas no está completamente llena, al apilar todas las capas se podría encontrar un espacio vacío en el cual entren mas cajas. Esta versión mejorada agrega una fase de relleno que intenta empaquetar cajas adicionales en ese espacio residual, permitiendo orientaciones mixtas para maximizar el espacio ocupado del contenedor.

La heurística opera en dos fases: primero ejecuta la heurística por capas estándar para establecer una configuración base de capas idénticas, y luego aplica un modelo tridimensional completo, similar al original, pero restringido para llenar los espacios vacíos restantes.

Fase 1: Empaquetamiento por capas base

Esta fase ejecuta exactamente el mismo algoritmo que la heurística por capas básica explicada anteriormente. El algoritmo `LayerHeuristic` itera sobre las seis orientaciones posibles, calcula el número de capas que caben para cada orientación, resuelve el problema bidimensional de una capa usando `CompleteModel2D`, y multiplica el resultado por el número de capas para obtener el total. Una vez probadas todas las orientaciones, se selecciona la que maximiza el número de cajas empaquetadas en el contenedor.

La diferencia principal respecto a la versión anterior es que ahora se conserva información adicional sobre la solución encontrada. Específicamente, además de contar cuántas cajas caben en una capa, el modelo bidimensional guarda las cajas específicas que forman parte de la solución mediante el método `getSelectedBoxes()`. Esta lista de cajas será crucial para la siguiente fase.

Fase 2: Relleno de espacios residuales

Una vez completada la fase de capas, el algoritmo procede a llenar el espacio residual que queda en el contenedor.

El primer paso es construir la lista de todas las cajas que se tienen en la solución por capas, ya que las cajas que fueron colocadas en la fase anterior eran solo de una capa. Esto se logra replicando la solución de la capa base en cada nivel vertical. Se obtiene la lista de cajas de la capa base mediante `bestModel.getSelectedBoxes()`, y luego se itera sobre cada una de las capas. Para cada caja en la capa base, se crea una nueva caja con los mismos índices i y j, pero con el índice k ajustado para el nivel de la capa actual (`k_original + zLayer`). Todas estas cajas se agregan a una lista llamada `fixed` que representa las cajas ya colocadas que no pueden moverse ni eliminarse.

Con la lista de cajas fijas, se crea una nueva instancia de la nueva versión de `CompleteModel`, el cual acepta una lista de cajas fijas. Este constructor recibe tres parámetros: la instancia del problema, la discretización del espacio, y la lista de cajas que ya están colocadas en el contenedor. El modelo resultante trata de encontrar la mejor configuración de cajas adicionales que puedan colocarse en el espacio residual sin interferir con las cajas ya existentes.

Modelo tridimensional con restricción de cajas fijas

Esta nueva versión del modelo `CompleteModel` ha sido modificada para manejar cajas fijas mediante un constructor que almacena la lista de cajas predefinidas en `_fixedBoxes`. Este modelo mantiene la misma estructura general que el modelo completo original: crea variables tridimensionales con todas las orientaciones posibles, define la función objetivo de maximización, y genera las restricciones de independencia y estabilidad.

La principal modificación sucede en el método `createVariables()`. Antes de crear una variable binaria para una posición (i,j,k,l), se verifica si la caja correspondiente se solapa con alguna de las cajas fijas, mediante el método auxiliar `overlapsFixed()` que determina si una caja propuesta comparte al menos un punto discreto del espacio con alguna caja de la lista. Esto asegura que el modelo de optimización sólo considere posiciones que no interfieran con las cajas ya colocadas en la fase de capas.

El método `overlapsFixed()` se basa en el método `contains()` de la clase `Box`. Para cada punto (i,j,k) de la discretización que está contenido dentro de la caja propuesta, se verifica si ese mismo punto está contenido dentro de alguna caja fija. Si existe al menos un punto compartido, significa que las cajas se solapan y la variable no debe crearse.

Las restricciones de estabilidad también se adaptan para considerar las cajas fijas. Además de contabilizar el soporte provisto por variables de decisión (cajas que el modelo puede decidir colocar o no), se agrega un término constante que representa el soporte provisto por las cajas fijas. Para cada caja variable en posición (i,j,k,l), se itera sobre todas las cajas fijas y se verifica si alguna tiene su techo al mismo nivel que el piso de la caja variable (`f.getTop() == box.getz()`). Si es así, se calcula la superficie de intersección entre el piso de la caja variable y el techo de la caja fija, y este valor se resta del lado izquierdo de la restricción de estabilidad como una constante. De esta manera, las cajas fijas contribuyen al soporte de las cajas variables, permitiendo que las nuevas cajas se apilen sobre la configuración base de capas.

Una vez completadas ambas fases, el algoritmo calcula las métricas finales. El número total de cajas es la suma de las cajas en la configuración base (cajas_por_capa × número_de_capas) más las cajas del relleno, obtenidas del valor objetivo del modelo residual.

12.2 Resultados

12.2.1 Contenedor 260x240x110

Para el contenedor de dimensiones 260x240x110, podemos ver que con el agregado de relleno 3D, la heurística mejora su desempeño respecto a la versión anterior. En la tabla 15 se observa que la cantidad de instancias en las que se aumentó la cantidad de cajas pasó de 1 a 3 , mientras que las instancias donde la solución se mantuvo igual aumentaron de 15 a 17. Consecuentemente, las instancias con disminución de

cajas se redujeron de 19 a 14. Esto indica que la fase de relleno logra recuperar parte del espacio que la heurística por capas básica desperdiciaba.

Tabla 15: Desempeño de la heurística por capas con relleno 3D respecto al método base para el contenedor 260x240x110

Resultado	Cantidad de instancias	Porcentualmente
Aumentó la cantidad de cajas	3	8,8%
La cantidad de cajas se mantuvo	17	50,0%
Disminuyó la cantidad de cajas	14	41,2%

En la tabla 16, podemos ver las 3 instancias en las que se obtuvo un mejor resultado que el caso base. En todas ellas, el tiempo de ejecución de la heurística es significativamente menor que el del modelo base, con aceleraciones que van desde 71x hasta 202x. Cabe destacar que la instancia (45,32,84), que ya era mejorada por la heurística sin relleno con 45 cajas, ahora alcanza las 60 cajas, lo que representa una mejora adicional de 15 cajas gracias a la fase de relleno.

Tabla 16: Instancias en las que la heurística por capas con relleno 3D aumentó la cantidad de cajas con respecto a la base para el contenedor 260x240x110.

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(41,24,113)	60	53	7,54	619,908	82,216
(45,32,84)	60	43	8,6	611,142	71,063
(45,30,84)	63	54	2,98	603,468	202,506

En cuanto a los tiempos de resolución, la tabla 17 muestra las 10 instancias donde la heurística alcanzó la misma solución que el modelo base en un tiempo menor. De estas instancias, 9 son óptimas y la instancia (45,35,61) es factible. Los factores de aceleración varían considerablemente, desde 1,21x hasta 81,9x, con un promedio de aproximadamente 25,3x. Esto muestra que la heurística es capaz de encontrar soluciones óptimas en tiempos significativamente menores para ciertos casos. Sin embargo, al incorporar una fase adicional con un modelo tridimensional, los tiempos de ejecución son mayores que los de la heurística por capas anterior, por ejemplo, la instancia (41,37,75) pasa de 1,2 segundos en la versión anterior a 4,84 segundos en la versión con relleno. Aun así, ambas versiones siguen siendo significativamente más rápidas que el modelo base.

Tabla 17: Instancias en las que el total de cajas se mantiene igual en la heurística por capas con relleno 3D y en el modelo base para el contenedor 260x240x110.

Caja	Total cajas	Estado	Tiempo heurística	Tiempo base	Aceleración
(30,30,85)	72	Optimal	0,86	1,039	1,208
(30,30,84)	72	Optimal	0,47	1,027	2,185
(40,43,84)	36	Optimal	2,11	28,237	13,382
(40,45,84)	36	Optimal	1,93	42,440	21,989
(40,37,75)	42	Optimal	4,03	253,392	62,876
(41,37,75)	42	Optimal	4,84	396,472	81,916
(45,35,61)	60	Feasible	9,86	646,780	65,596
(50,45,109)	25	Optimal	1,15	2,809	2,443
(55,40,80)	36	Optimal	0,63	1,360	2,159
(125,19,118)	20	Optimal	0,75	1,137	1,516

Adicionalmente, las 7 instancias restantes en las que la heurística encuentra la misma solución que el caso base tienen un tiempo de resolución mayor. Sin embargo, la diferencia es menor a 0,1 segundos en todos los casos, lo cual resulta despreciable en términos prácticos.

Con esta heurística también encontramos el trade-off entre calidad de la solución y velocidad. En la tabla 18 se muestran 2 instancias donde la heurística encontró soluciones con menor cantidad de cajas que el caso base.

Tabla 18: Instancias resueltas con la heurística por capas con relleno 3D con mayores aceleraciones para el contenedor 260x240x110.

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(135,29,47)	25	32	4,97	604,216	121,573
(135,29,46)	27	32	5,63	604,582	107,386

A diferencia de la heurística por capas básica, donde estas mismas instancias presentaban pérdidas del 50% (16 cajas frente a 32 del caso base), la versión con relleno logra recuperar parte de ese espacio, alcanzando 25 y 27 cajas respectivamente. Esto representa una mejora de 9 y 11 cajas gracias a la fase de relleno, aunque aún con pérdidas del 22% y 16% respecto al caso base. Las aceleraciones se mantienen superiores a 100x, lo que indica que incluso en los casos donde la heurística no alcanza la solución óptima, ofrece un compromiso razonable entre calidad y velocidad. Comparando ambas versiones, se evidencia que la fase de relleno mitiga significativamente el trade-off.

12.2.2 Contenedor 400x240x130

En el contenedor con dimensiones 400x240x130, la heurística encontró mejores soluciones en 13 instancias en comparación al caso base, 9 instancias más que en la heurística por capas sin relleno 3D. En la tabla 19 se observa que las instancias donde aumentó la cantidad de cajas pasaron de 4 a 13, mientras que las instancias con disminución se redujeron de 25 a 12. Este cambio evidencia nuevamente el impacto positivo de la fase de relleno.

Tabla 19: Desempeño de la heurística por capas con relleno 3D respecto al método base para el contenedor 400x240x130.

Resultado	Cantidad de instancias	Porcentualmente
Aumentó la cantidad de cajas	13	38,2%
La cantidad de cajas se mantuvo	9	26,5%
Disminuyó la cantidad de cajas	12	35,3%

En la tabla 20 podemos ver las 13 instancias en las que se obtuvo un mejor resultado que el caso base. En todas ellas, el tiempo de ejecución de la heurística es significativamente menor que el del modelo base, con un factor de aceleración promedio de 17,6x. Cabe destacar que en 3 de estas instancias, (40,37,75), (41,37,75) y (45,35,61), el modelo base no logró encontrar ninguna solución dentro del límite de tiempo, mientras que la heurística encontró soluciones de 120, 120 y 110 cajas respectivamente.

Tabla 20: Instancias en las que la heurística por capas con relleno 3D aumentó la cantidad de cajas con respecto a la base para el contenedor 400x240x130.

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(40,35,75)	132	99	8,310	618,844	74,470
(40,43,84)	103	74	71,070	718,756	10,113
(40,45,84)	88	74	104,270	692,304	6,639
(40,37,75)	120	0	261,220	966,653	3,700
(41,37,75)	120	0	289,740	1.094,801	3,779
(41,24,113)	129	89	140,780	700,830	4,978
(45,35,61)	110	0	125,730	1.473,243	11,718
(45,32,84)	112	90	96	779,438	8,119
(45,30,84)	112	95	16,120	632,769	39,254
(135,29,47)	60	49	100,330	647,342	6,452
(135,29,46)	60	51	102,580	667,635	6,508
(138,30,47)	60	49	94,870	666,952	7,030
(160,33,46)	50	41	25,120	616,901	24,558

En cuanto a los tiempos de resolución, la tabla 21 muestra las 5 instancias donde la heurística alcanzó la misma solución que el modelo base en un tiempo menor. Los factores de aceleración varían desde 5,21x hasta 43,39x, con un promedio de aproximadamente 22,6x. Si bien la aceleración promedio en este contenedor es menor que en el contenedor anterior (22,6x vs 25,3x), estos resultados siguen demostrando que la heurística es capaz de encontrar soluciones óptimas en tiempos significativamente menores. Nuevamente, los tiempos son mayores que en la heurística por capas sin relleno debido a la fase adicional del modelo tridimensional, pero aún siguen siendo considerablemente menores que el caso base.

Tabla 21: Instancias en las que el total de cajas se mantiene igual en la heurística por capas con relleno 3D y en el modelo base para el contenedor 400x240x130.

Caja	Total cajas	Estado	Tiempo heurística	Tiempo base	Aceleración
(40,40,75)	90	Optimal	0,64	12,211	19,079
(40,40,76)	90	Optimal	0,79	4,112	5,205
(50,45,109)	40	Optimal	2,95	86,058	29,172
(125,19,118)	42	Optimal	4,76	78,761	16,546
(55,40,80)	63	Optimal	1,01	43,825	43,391

Adicionalmente, las 4 instancias restantes en las que la heurística encuentra la misma solución que el caso base tienen un tiempo de resolución mayor. Sin embargo, las diferencias son pequeñas, menores a 0,35 segundos en todos los casos, lo cual resulta despreciable en términos prácticos.

En este contenedor también se presenta el trade-off entre calidad de la solución y velocidad. En la tabla 22 se muestran 4 instancias donde la heurística encontró soluciones con menor cantidad de cajas que el caso base pero con tiempos significativamente menores.

Tabla 22: Instancias resueltas con la heurística por capas con relleno 3D con mayores aceleraciones para el contenedor 400x240x130

Caja	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(30,30,85)	148	149	1,51	20,655	13,679
(30,30,84)	148	149	1,49	28,492	19,122
(62,39,92)	48	50	3,64	301,127	82,727
(64,39,93)	48	50	4,34	328,600	75,714

A diferencia de la heurística por capas básica, donde las mayores aceleraciones venían acompañadas de pérdidas significativas de hasta el 50% de las cajas, en esta versión con relleno las degradaciones son menores. En todos estos casos, la diferencia es de solo 1 o 2 cajas, lo que representa pérdidas menores al 4%. Esto indica que la fase de relleno mitiga considerablemente el compromiso entre velocidad y calidad en donde se mantienen aceleraciones importantes pero con un impacto mínimo en la cantidad de cajas empaquetadas.

12.3 Ventajas y Limitaciones

La principal ventaja de esta heurística es que combina la eficiencia computacional del enfoque bidimensional con la flexibilidad del modelo tridimensional completo. La fase de capas resuelve la mayor parte del problema de manera rápida, mientras que la fase de relleno aprovecha el espacio residual sin comprometer significativamente el tiempo de ejecución. Adicionalmente, la fase de relleno permite orientaciones mixtas por lo que proporciona mayor flexibilidad para aprovechar espacios irregulares. De esta manera, se logra resolver la principal limitación de la heurística anterior.

Los resultados demuestran estas ventajas. En el contenedor 260x240x110, las instancias donde aumentó la cantidad de cajas pasaron de 1 a 3, y las instancias con disminución se redujeron de 19 a 14. En el contenedor 400x240x130, la mejora es aún más notable, las instancias donde aumentó la cantidad de cajas pasaron de 4 a 13, mientras que las instancias con disminución se redujeron de 25 a 12. Esto evidencia que la fase de relleno logra recuperar una parte significativa del espacio que la heurística por capas básica

desperdiciaba, especialmente en contenedores de mayor tamaño donde el espacio residual aprovechable es más significativo.

En cuanto al trade-off entre calidad y velocidad, la heurística con relleno presenta un balance más favorable que la versión básica. En la heurística por capas sin relleno, las mayores aceleraciones venían acompañadas de pérdidas significativas de hasta el 50% de las cajas. En esta versión, las degradaciones son considerablemente menores. Por ejemplo, en el contenedor 400x240x130, las instancias con peor desempeño presentan diferencias de solo 1 o 2 cajas respecto al caso base con aceleraciones de hasta 82x. En el contenedor 260x240x110, las instancias (135,29,47) y (135,29,46), que en la versión básica presentaban pérdidas del 50%, ahora alcanzan 25 y 27 cajas respectivamente frente a las 16 de la versión sin relleno, reduciendo la brecha con el caso base sin sacrificar la eficiencia computacional.

Sin embargo, la heurística sigue presentando algunas limitaciones. La restricción de que todas las cajas en las capas base deben tener la misma orientación sigue siendo una simplificación que podría no ser óptima. Además, la descomposición en dos fases es una decisión greedy: la configuración de capas base se fija antes de considerar el relleno, lo que podría impedir encontrar configuraciones mejores donde una orientación base diferente, la cual podría tener menos cajas por capa pero más espacio residual, podría resultar en un total superior. Los resultados muestran que, a pesar de las mejoras, la heurística aún encuentra soluciones con menor cantidad de cajas que el caso base en el 41,2% de las instancias del contenedor 260x240x110 y en el 35,3% del contenedor 400x240x130. Finalmente, la fase de relleno resuelve un problema de optimización completo sobre el espacio residual, lo que puede ser computacionalmente costoso si el espacio residual es grande.

13. Heurística 4: Agrupando cajas

13.1 Algoritmo

La heurística de agrupamiento de cajas en bloques surge a partir de la observación de que, cuanto más similares son las dimensiones de la caja a las del contenedor, menor es el tiempo de resolución del problema, ya que existen menos combinaciones posibles de ordenamiento. Con esta idea, se propuso preagrupar las cajas en bloques de mayor volumen antes de iniciar la construcción de la solución, y posteriormente resolver el posicionamiento de estos bloques dentro del contenedor. Lo que sería equivalente a trabajar con cajas de mayor volúmen

Fase 1: Definición de instancias y tipos de bloques

Se modificó el archivo `EntryPoint`, para que cree un conjunto de instancias con un contenedor de dimensiones fijadas antes de comenzar {L, W, H} y el conjunto de tamaños de cajas {l, w, h}. Para cada combinación {L, W, H, l, w, h} se crea el objeto Instance, que almacena las seis dimensiones.

El paso más importante de esta fase es la creación de los tipos de bloques mediante el método `buildBlocks()`. En este proceso se utilizan objetos de la clase `BlockType`, que representan agrupaciones de cajas idénticas tratadas como una sola unidad. Cada `BlockType` se define a partir de una tupla {a, b, c}, donde 'a' indica la cantidad de cajas alineadas a lo largo (eje X), 'b' la cantidad a lo ancho (eje Y) y 'c' la cantidad en altura (eje Z). A partir de estos valores y de las dimensiones de la caja obtenidas de la instancia del problema, la clase calcula automáticamente las dimensiones reales del bloque, es decir, su largo, ancho y alto totales. Esto es necesario para que el modelo conozca el tamaño exacto que ocupará cada bloque dentro del contenedor y pueda decidir su ubicación sin superposiciones. De este modo, el método `buildBlocks()` genera distintas combinaciones posibles que luego el modelo intentará colocar.

Durante el desarrollo se ensayaron diferentes configuraciones de bloques y órdenes de resolución, a fin de evaluar el impacto del tamaño y la composición de los bloques sobre la calidad y el tiempo de las soluciones obtenidas. En todas ellas se incluyó también el bloque 1×1×1 al final, que representa una única caja individual, para que el algoritmo evalúe si pueden ubicarse cajas sueltas en los espacios restantes. Las estrategias fueron:

- Estrategia 1 – Bloques grandes a pequeños (genéricos): Se comienza agrupando cajas en bloques grandes y, en sucesivas iteraciones, se reduce su tamaño para completar los espacios residuales. El proceso finaliza con cajas individuales. Se utilizan bloques genéricos: {3,2,1}, {2,2,1}, {2,1,2}, {1,2,2}, {2,1,1}, {1,1,2}, {1,1,1}.
- Estrategia 2 – Bloques medianos a pequeños (genéricos): Similar a la anterior, pero iniciando con bloques medianos y reduciendo progresivamente el tamaño hasta llegar a las unidades. Bloques utilizados: {2,2,1}, {3,1,1}, {2,1,1}, {1,1,2}, {1,1,1}.
- Estrategia 3 – Bloques grandes adaptados al contenedor: Parte desde bloques grandes y va disminuyendo su tamaño, pero los conjuntos de bloques se diseñan según la proporción del contenedor. Por ejemplo, en contenedores anchos se priorizan bloques más planos y extendidos.
    - Contenedor 1: {3,2,2}, {2,3,2}, {2,2,2}, {3,2,1}, {2,3,1}, {2,2,1}, {3,1,1}, {2,1,1}, {1,2,1}, {1,1,2}, {1,1,1}
    - Contenedor 2: {3,3,2}, {3,2,2}, {2,3,2}, {3,2,1}, {2,3,1}, {2,2,2}, {2,2,1}, {3,1,1}, {2,1,1}, {1,2,1}, {1,1,2}, {1,1,1}.
- Estrategia 4 – Bloques medianos adaptados al contenedor: Igual criterio de adaptación al contenedor que en la estrategia 3, pero comenzando desde bloques de tamaño medio.
    - Contenedor 1: {2,2,2}, {3,2,1}, {2,3,1}, {2,2,1}, {3,1,1}, {2,1,1}, {1,2,1}, {1,1,2}, {1,1,1}
    - Contenedor 2: {2,2,2}, {3,2,1}, {2,3,1}, {2,2,1}, {3,1,1}, {2,1,1}, {1,2,1}, {1,1,2}, {1,1,1}.
- Estrategia 5 – Bloques planos de dos cajas: Se agrupan únicamente bloques planos formados por dos cajas ({2×1×1}, {1×2×1}, {1×1×2}) y luego se intenta completar con cajas individuales en los huecos restantes.

- Estrategia 6 – Bloques planos de tres cajas. Comienza con bloques planos de tres cajas ({3×1×1}, {1×3×1}, {1×1×3}), continúa con los de dos cajas y finaliza con cajas individuales.

Fase 2: Discretización del espacio

Posteriormente, y antes de buscar la solución, se construye la representación discreta del espacio interno del contenedor mediante la llamada a `Discretization()`, que convierte el espacio continuo en una grilla tridimensional (calculada en función del tamaño de la caja particular y del contenedor). El objeto generado, disc, se pasa inmediatamente a `BlockModelIterative()` que lo utiliza para generar los puntos factibles de colocar un bloques, chequear si un bloque efectivamente entra en el contenedor, calcular solapamientos de cajas y marcar espacios ocupados en el contenedor cuando se decide colocar una bloque.

Fase 3: Ejecución del modelo iterativo

Una vez generada la lista de bloques y discretizado el espacio, se crea un objeto de tipo `BlockModelIterative`, que constituye el núcleo de la heurística de agrupamiento. El método `solve()` de esta clase implementa una estrategia iterativa que recorre la lista de tipos de bloque, resolviendo en cada iteración un modelo de optimización independiente y acumulando los resultados parciales. En cada paso, se instancia un modelo en el que se definen variables binarias `x[i][j][k][p]` que representan la colocación posible de un bloque en la posición discreta (i, j, k) y con una orientación determinada, tomando las coordenadas físicas a partir del objeto disc.

El modelo incluye restricciones que garantizan el no solapamiento entre bloques y la estabilidad estructural, imponiendo que cada bloque cuente con al menos un 70 % de su base apoyada sobre otros bloques o sobre el piso. Este soporte se calcula combinando la contribución de los bloques previamente colocados y la de los que se ubican en la iteración actual. Una vez resuelto el submodelo, los bloques seleccionados se registran como objetos `PlacedBlock`, se incrementa el contador `totalBoxesPlaced`, y se actualiza la máscara tridimensional `occupiedMask`, que indica las regiones ya ocupadas del contenedor. Este proceso se repite para todos los tipos de bloque, construyendo una solución factible de forma incremental, reutilizando el espacio disponible y reduciendo la complejidad computacional del modelo completo.

Fase 4: Salida y registro

Una vez finalizado `bm.solve()`, el método `runInstance()` mide el tiempo total (tiempo) de resolución desde que se llama al `BlockModelIterative`, y obtiene el estado de resolución (status) que mide la optimalidad de la solución construída dentro del esquema heurístico y la cantidad de cajas colocadas (`bm.getTotalBoxesPlaced()`) calculada como el producto entre la cantidad de bloques colocados de cada tamaño y la cantidad de cajas en cada tipo de bloque. Estos valores se registran en el

archivo `resultados.csv`, una línea por cada tipo de caja. De esta manera, el programa puede evaluar comparativamente cuántas cajas se logran empaquetar con la heurística de bloques frente a otros enfoques.

13.2 Resultados

13.2.1 Contenedor 260x240x110

Como supusimos al comienzo; la seguidilla puntual de bloques que cada estrategia utiliza impacta directamente en el resultado de la función objetivo y en el tiempo de resolución para cada instancia específica. Esto se debe a que cada combinación de contenedor y caja define una geometría particular, y por lo tanto no todas las configuraciones de bloques son igualmente favorables para todas las instancias.

Por este motivo se presentan, primeramente, los resultados agregados; el foco no estuvo en encontrar una estrategia "óptima" para todos los tamaños de cajas, sino evaluar sistemáticamente si alguna de las estrategias logra alcanzar una solución de mejor calidad para cada instancia individual.

Tabla 23: Resultados agrupados por mejoría para la heurística 4, contenedor 260x240x110.

Resultado	Cantidad de instancias	Porcentualmente
Con alguna estrategia aumentó la cantidad de cajas	2	5,71%
Con alguna estrategia la cantidad de cajas se mantuvo	22	62,86%
Para todas las estrategias disminuyó la cantidad de cajas	11	31,43%

En la Tabla 24 se desglosan los resultados de arriba a cada estrategia, en pos de observar más en detalle el comportamiento individual de cada una y entender cómo contribuyó a los patrones generales comentados antes.

Se puede ver que para este primer contenedor ninguna estrategia logra generar mejoras en una porción significativa de las instancias; en términos de cantidad de cajas las estrategias #1 y #4 no alcanzan ninguna mejoría, la #2, #3 y #6 mejoran en una sola instancia, y la número #5 mejora para dos tamaños de cajas. Se considera importante destacar que las instancias en las que la heurística alcanza una mejor respuesta, en el modelo base se veían limitadas por el tiempo, llegando a una respuesta parcial en, por lo menos, 600 segundos.

Tabla 24: Comparación agrupada de resultados según su mejora, por estrategia de agrupación para la heurística 4, contenedor 260x240x110.

Estrategia	Aumentó la cantidad de cajas	Porcentualmente	La cantidad de cajas se mantuvo	Porcentualmente	Disminuyó la cantidad de cajas	Porcentualmente
#1	0	0%	16	45,71%	19	54,29%
#2	1	2,86%	14	40%	20	57,14%
#3	1	2,86%	15	42,86%	19	54,3%
#4	0	0%	15	42,86%	20	57,14%
#5	2	5,71%	19	54,29%	14	40%
#6	1	2,86%	15	42,86%	19	54,29%

En la Tabla 25 se muestra el valor que alcanzó la función objetivo, en qué tiempo lo hizo y con qué estrategia, para dichas instancias. Se puede ver que para estas instancias la cantidad de cajas aumentó entre 1 y 3 unidades. Lo más llamativo es la comparación del tiempo que logra acelerarse entre 8% y 30%, promediando una aceleración del 17,115% aproximadamente. Estas instancias que antes caían en un timeout, con el uso de la heurística de agrupar en bloques de cajas alcanza mejores respuestas en menos tiempo.

Tabla 25: Desglose de resultados por instancia en que se consiguió una mejor respuesta y comparación con modelo base para la heurística 4, contenedor 260x240x110.

Caja	Estrategia	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(45, 32, 84)	#2	44	43	19,847	611,142	30,793
(45, 32, 84)	#3	46	43	33,028	611,142	18,504
(41, 24, 113)	#5	56	53	73,306	619,908	8,456
(45, 32, 84)	#5	46	43	73,444	611,142	9,321
(45, 32, 84)	#5	46	43	33,028	611,142	18,504

Por otro lado, en la Tabla 26 se enuncian para cada estrategia métricas asociadas a los tiempos obtenidos en las instancias en que la cantidad de cajas se mantuvo. Se dividió para cada estrategia en aquellas instancias dónde se mantuvo la respuesta óptima, y aquellas en las que se mantuvo una respuesta factible. Se puede ver que la aplicación de la heurística (con alguna de sus estrategias) permite mantener

respuestas óptimas acelerando el tiempo de búsqueda en promedio 6,901 veces, y respuestas factibles en promedio 20,631 veces. Es decir que, de acuerdo a estas observaciones, se podría mantener la calidad de aproximadamente el 45% de las instancias acelerando sus tiempos de resolución en promedio 13,191 veces.

Tabla 26: Comparación agrupada de los resultados de las instancias en que se mantiene la respuesta con el modelo base para la heurística 4, contenedor 260x240x110.

Estrategia		Cantidad	Tiempo min. base	Tiempo min. heur.	Tiempo max. base	Tiempo max. heur.	Tiempo prom. base	Tiempo prom. heur.	Aceleración prom.
#1	**Óptimos mantenidos**	15	0,123	0,009	42,440	12,249	5,273	1,715	3,0746
	Factibles mantenidos	1	646,780	61,195	646,780	61,195	646,780	61,195	10,569
#2	**Óptimos mantenidos**	14	0,123	0,007	42,440	9,176	5,6	1,428	3,922
	Factibles mantenidos	0	-	-	-	-	-	-	-
#3	**Óptimos mantenidos**	12	0,142	0,023	42,440	4,206	6,560	0,696	9,425
	Factibles mantenidos	3	604,216	34,446	646,780	102,66	618,526	58,039	10,657
#4	**Óptimos mantenidos**	13	0,142	0,017	42,440	4,385	6,070	0,567	10,705
	Factibles mantenidos	2	604,216	25,712	604,582	28,595	604,399	27,154	22,258
#5	**Óptimos mantenidos**	17	0,123	0,013	396,472	81,125	42,885	8,833	4,855
	Factibles mantenidos	2	604,216	11,772	604,582	12,889	604,399	12,331	49,015
#6	**Óptimos mantenidos**	12	0,142	0,023	42,440	4,206	6,56	0,696	9,425
	Factibles mantenidos	3	604,216	34,446	646,780	102,66	618,526	58,039	10,657

Siguiendo con lo enunciado en el párrafo anterior, en la Tabla 27 se explica el detalle de los casos por estrategia en los que la heurística fue más rápida que el modelo base, pero la función objetivo alcanzó una cifra menor. Los resultados muestran que, aún cuando la heurística sacrifica entre, aproximadamente, 2,4 y 4,7 cajas de la solución, ese deterioro relativamente pequeño en la función objetivo permite obtener una aceleración significativa de los tiempos de resolución. Según la estrategia considerada, la heurística logra acelerar la resolución de las instancias entre 5,874 y 32,150 veces. En otras palabras, aceptar una reducción promedio de 4.171 cajas de la función objetivo, permite alcanzar soluciones en un tiempo promedio 15.563 veces más veloz.

Este comportamiento evidencia un trade-off claro entre la calidad de la solución y el tiempo de computo. En promedio en un 53% de las instancias la heurística logra soluciones ligeramente peores que el modelo base, pero a cambio logra una reducción bastante significativa de los tiempos de resolución. En contextos donde el tiempo de respuesta es un recurso crítico, este compromiso puede ser claramente preferible frente a la búsqueda de la mejor solución posible desde el punto de vista estrictamente óptimo.

Tabla 27: Medición agrupada de la reducción de la calidad de la respuesta y comparativa de tiempo con modelo base para la heurística 4, contenedor 260x240x110.

Estrategia	Cantidad	Promedio de cajas en que se redujo la solución	Tiempo promedio base	Tiempo promedio heurística	Aceleración
#1	19	4,578947368	202,248	6,291	32,151
#2	20	3,9	194,011	7,499	25,871
#3	19	4,684210526	106,543	11,557	9,219
#4	20	4,75	164,103	14,853	11,048
#5	14	2,428571429	100,062	17,035	5,874
#6	19	4,684210526	106,543	11,557	9,219

13.2.2 Contenedor 400x240x130

En este segundo contenedor, los resultados requieren una lectura diferente debido a la superposición natural entre las categorías. Dado que una instancia puede mejorar en una estrategia y, al mismo tiempo, mantenerse igual en otra, los conjuntos “Con alguna estrategia aumentó la cantidad de cajas” y “Con alguna estrategia la cantidad de cajas se mantuvo” no son excluyentes entre sí, por lo que sus cantidades no necesariamente suman el total de instancias. Esta superposición es inherente al diseño del experimento; cada estrategia opera con una secuencia distinta de bloques y, por lo tanto, una misma instancia puede

reaccionar de manera diferente según el orden considerado. Por este motivo, la Tabla 28 presenta las tres métricas relevantes de forma independiente.

Vemos que en el segundo contenedor la heurística muestra un desempeño más favorable, ya que once instancias logran mejorar respecto del modelo base y, además, se mantiene un número significativo de casos en los que al menos una estrategia iguala la solución base, reduciendo en 6 instancias. Mientras que la cantidad de cajas para las que todas las heurísticas encuentran una peor solución aumentó en tal sólo una unidad. Este contraste sugiere que, como bien hipotetizamos, la efectividad relativa del enfoque por bloques no es uniforme, sino que depende de la geometría específica del contenedor y de cómo ésta interactúa con las configuraciones de bloques disponibles.

Tabla 28: Resultados agrupados por mejoría para la heurística 4, contenedor 400x240x130.

Resultado	Cantidad de instancias	Porcentualmente
Con alguna estrategia aumentó la cantidad de cajas	11	31,43%
Con alguna estrategia la cantidad de cajas se mantuvo	16	62,86%
Para todas las estrategias disminuyó la cantidad de cajas	12	34,29%

En este segundo contenedor podemos ver que, si bien, para todas las estrategias se observa una reducción en la cantidad de cajas de la función objetivo, hay un patrón más equilibrado entre los tres fenómenos señalados. Estrategias como la #1, que en el primer contenedor no lograban mejorar el resultado de ninguna instancia, en este contenedor sí lo hacen. Resulta particularmente llamativo el caso de la #4, que en el primer contenedor estaba en 0, y en este contenedor alcanza a mejorar 8 instancias diferentes. Se supone, como posible explicación a este comportamiento, que la geometría más amplia (particularmente el mayor largo y la mayor altura útil) ofrece más posiciones viables para los bloques medianos con los que esta estrategia inicia. En el primer contenedor, esos bloques tienden a generar configuraciones rígidas que luego resultan difíciles de completar, mientras que en el segundo contenedor la heurística cuenta con mayor margen para acomodarlos sin perjudicar el espacio remanente. Además, varias de las cajas que mejoran en este segundo contenedor tienen una dimensión marcadamente menor, lo que favorece el encastre de bloques anchos, coherente con la lógica de construcción de esta estrategia.

Tabla 29: Comparación agrupada de resultados según su mejora, por estrategia de agrupación para la heurística 4, contenedor 400x240x130.

Estrategia	Aumentó la cantidad de cajas	Porcentualmente	La cantidad de cajas se mantuvo	Porcentualmente	Disminuyó la cantidad de cajas	Porcentualmente
#1	4	11,43%	7	20%	24	68,57%
#2	5	14,29%	5	14,29%	25	71,43%
#3	8	22,86%	7	20%	20	57,14%
#4	8	22,86%	4	11,43%	23	65,71%
#5	1	2,86%	12	34,29%	22	62,86%
#6	4	11,43%	9	25,71%	22	62,86%

En la Tabla 30 se detallan las instancias en las que la heurística logró superar al modelo base para el segundo contenedor, indicando para cada una el valor de la función objetivo alcanzado, el tiempo de resolución y la estrategia responsable de la mejora. A diferencia del contenedor anterior (donde los aumentos eran más sutiles), en este caso las diferencias a favor de la heurística parecen ser sustanciales. En casos donde no se lograba colocar ninguna caja con el modelo base, la heurística aumenta esta cantidad en valores que van desde las 88 cajas a las 108 (con un promedio de 94 cajas). Mientras que en instancias en las que el modelo base lograba colocar cajas, los aumentos van entre 1 y 15 cajas (promediando las 5 cajas extra).

Otro aspecto destacable es el comportamiento temporal. Si bien el tiempo de la heurística no es necesariamente bajo para todas las estrategias, la comparación con el tiempo base revela aceleraciones muy pronunciadas. En la mayoría de los casos, el modelo base requiere tiempos cercanos o superiores al minuto, mientras que la heurística obtiene mejores soluciones con aceleraciones que, según la instancia, pueden oscilar entre reducciones moderadas y reducciones muy marcadas del tiempo de cómputo. Logrando una aceleración promedio para estas instancias de 3,51x.

Tabla 30: Desglose de resultados por instancia en que se consiguió una mejor respuesta y comparación con modelo base para la heurística 4, contenedor 400x240x130.

Caja	Estrategia	Total cajas heurística	Total cajas base	Tiempo heurística	Tiempo base	Aceleración
(40, 37, 75)	#1	90	0	602,665	966,653	1,604
(41, 37, 75)	#1	90	0	777,988	1094,801	1,407
(41, 24, 113)	#1	102	89	282,012	700,830	2,485
(45, 35, 61)	#1	106	0	949,58	1473,243	1,551
(40, 43, 84)	#2	80	74	153,493	718,756	4,683
(40, 37, 75)	#2	90	0	501,415	966,653	1,929
(41, 37, 75)	#2	88	0	565,162	1094,801	1,937
(41, 24, 113)	#2	97	89	235,75	700,830	2,973
(45, 30, 84)	#2	98	95	50,265	632,769	12,589
(40, 43, 84)	#3	80	74	270,97	718,756	2,656
(40, 37, 75)	#3	90	0	923,97	966,653	1,046
(41, 37, 75)	#3	90	0	1061,779	1094,801	1,031
(41, 24, 113)	#3	96	89	239,934	700,830	2,921
(45, 35, 61)	#3	108	0	1526,685	1473,243	0,965
(135, 29, 47)	#3	52	49	200,446	647,342	3,23
(135, 29, 46)	#3	52	51	288,454	667,635	2,315
(138, 30, 47)	#3	51	49	242,102	666,952	2,755
(40, 43, 84)	#4	80	74	311,016	718,756	2,311
(40, 45, 84)	#4	75	74	316,183	692,304	2,190
(40, 37, 75)	#4	89	0	940,621	966,653	1,028
(41, 37, 75)	#4	88	0	1051,838	1094,801	1,041
(41, 24, 113)	#4	104	89	202,193	700,830	3,466

(45, 35, 61)	#4	102	0	1447,22	1473,243	1,018
(45, 30, 84)	#4	96	95	52,628	632,769	12,023
(160, 33, 46)	#4	43	41	54,490	616,901	11,321
(45, 30, 84)	#5	97	95	156,129	632,769	4,0529
(41, 24, 113)	#6	96	89	263,038	700,830	2,664
(135, 29, 47)	#6	51	49	133,619	647,342	4,845
(135, 29, 46)	#6	56	51	132,110	667,635	5,054
(138, 30, 47)	#6	51	49	107,729	666,952	6,191

La Tabla 31 presenta, para cada estrategia, el comportamiento temporal en las instancias donde la heurística logró mantener la calidad de la solución (tanto óptimos como respuestas factibles), para así evaluar cómo se comporta cada enfoque cuando no hay variación en la función objetivo. En términos generales, el patrón predominante es claro; cuando la heurística mantiene la calidad de la solución, suele hacerlo en tiempos sensiblemente menores que el modelo base.

En los casos donde se preserva el valor óptimo, varias estrategias muestran tiempos mínimos y máximos de la heurística muy inferiores a los del modelo base. Esto se refleja en que la aceleración promedio de estas instancias al aplicar el modelo heurístico fue de 5,028. Este comportamiento refuerza la idea de que, aun cuando no produce mejoras, la heurística es capaz de reconstruir soluciones óptimas con un costo computacional mucho menor, gracias a que reduce el espacio de búsqueda de forma más estructurada.

El mismo fenómeno aparece en las instancias en las que se mantuvo una respuesta factible. En los casos en los que pudo reducir el tiempo de resolución promedio, los rangos en los que se estabilizan estos tiempos son considerablemente menores. Los tiempos de resolución se aceleran en promedio para estas instancias 9,098 veces (con picos de aumento de hasta 29 veces). No obstante, es importante mencionar que en tres de las seis estrategias, para las instancias en que se mantiene una solución factible, el tiempo promedio en el que se resolvió aplicando la heurística aumentó en relación al base; estos casos sirven para remarcar que el enfoque heurístico, si bien eficiente en promedio, puede no ser siempre la alternativa más veloz para todas las configuraciones.

Tabla 31: Comparación agrupada de los resultados de las instancias en que se mantiene la respuesta con el modelo base para la heurística 4, contenedor 400x240x130.

Estrategia		Cantidad	Tiempo min. base	Tiempo min. heur.	Tiempo max. base	Tiempo max. heur.	Tiempo prom. base	Tiempo prom. heur.	Aceleración prom.
#1	**Óptimos mantenidos**	6	0,153	0,042	86,058	15,728	17,155	2,99	5,741
	Factibles mantenidos	1	618,844	21,36	618,844	21,36	618,844	21,36	28,972
#2	**Óptimos mantenidos**	3	0,153	0,037	0,243	0,098	0,183	0,0613	2,989
	Factibles mantenidos	2	692,304	142,088	1473,24	2643,089	1082,8	1392,59	0,778
#3	**Óptimos mantenidos**	6	0,153	0,125	12,211	1,259	2,860	0,542	5,271
	Factibles mantenidos	1	618,844	28,578	618,844	28,578	618,844	28,578	21,655
#4	**Óptimos mantenidos**	3	0,153	0,049	0,243	0,147	0,183	0,093	1,978
	Factibles mantenidos	1	779,438	355,473	779,438	355,473	779,438	355,473	2,193
#5	**Óptimos mantenidos**	9	0,154	0,034	328,6	46,061	88,002	14,939	5,892
	Factibles mantenidos	3	966,653	1920,40	1473,24	2749,26	1178,2	2350,8	0,501
#6	**Óptimos mantenidos**	5	0,153	0,04	86,058	16,227	20,538	6,285	3,268
	Factibles mantenidos	4	618,844	115,254	1473,24	3599,12	1038,3	2110,21	0,492

Por último, la Tabla 32 resume, para cada estrategia, el comportamiento de la heurística en las instancias donde la cantidad de cajas disminuyó respecto del modelo base. Aun tratándose de casos en los que la calidad de la solución empeora, se observa un patrón consistente; el deterioro promedio en la función objetivo es acotado, mientras que la reducción en los tiempos de resolución es, en la mayoría de las estrategias, marcadamente superior. Las estrategias muestran comportamientos distintos (algunas reducen

sólo unas pocas cajas en promedio, y otras presentan descensos algo mayores) pero en todos los casos la magnitud del deterioro es relativamente pequeña frente al ahorro de tiempo obtenido.

En otras palabras, aceptar una reducción promedio de 6,255 cajas en la función objetivo implica un deterioro acotado en la calidad de la solución, pero habilita un beneficio computacional considerable; en promedio, la heurística permite resolver estas instancias hasta 4,462 veces más rápido que el modelo base. Este intercambio evidencia que, aun cuando la heurística no iguala la calidad óptima, el ahorro de tiempo obtenido puede justificar plenamente su uso en escenarios donde la velocidad de cálculo resulta un factor determinante.

Tabla 32: Medición agrupada de la reducción de la calidad de la respuesta y comparativa de tiempo con modelo base para la heurística 4, contenedor 400x240x130.

Estrategia	Cantidad	Promedio de cajas en que se redujo la solución	Tiempo promedio base	Tiempo promedio heurística	Aceleración
#1	24	5,417	264,235	48,779	5,417
#2	25	3,56	200,762	30,168	6,655
#3	20	5,25	186,337	53,792	3,464
#4	23	4,348	157,509	24,247	6,496
#5	22	14,818	156,141	288,157	1,845
#6	22	4,136	198,180	68,492	2, 893

13.3 Ventajas y Limitaciones

Los resultados obtenidos en ambos contenedores muestran que la heurística de agrupamiento en bloques aporta beneficios consistentes en términos de tiempo de cómputo, incluso en escenarios donde no se logra mejorar la función objetivo. En promedio, cuando la heurística mantiene la calidad de la solución (ya sea óptima o factible) lo hace entre 1 y 50 veces más rápido que el modelo base, con picos considerablemente superiores en instancias difíciles. Esto se explica por la reducción drástica del espacio de búsqueda; trabajar con bloques preconfigurados permite explorar combinaciones estructurales sin realizar exhaustivamente todas las colocaciones posibles una por una, como sí lo hace el modelo base.

Además, cuando la heurística logra mejorar la cantidad de cajas ubicadas, lo hace de forma contundente. En el segundo contenedor, por ejemplo, las mejoras van desde incrementos de 1 a 15 cajas en instancias donde el modelo exacto colocaba apenas unas pocas, e incluso aparecen casos extremos donde el modelo

base no logra colocar ninguna caja y la heurística ubica entre 80 y 108 unidades. Esto sugiere que la heurística es particularmente efectiva cuando la geometría del contenedor es compatible con la forma de los bloques iniciales, ya que logra explotar configuraciones estructurales que el modelo exacto no alcanza dentro de tiempos razonables.

Una última ventaja relevante es su tendencia a estabilizar los tiempos de ejecución. A diferencia del modelo base, cuyos tiempos varían enormemente entre instancias (y en muchos casos trepan hacia varios cientos o miles de segundos), la heurística tiende a fijarse en rangos acotados, incluso en configuraciones complejas. Esto la vuelve especialmente adecuada para escenarios donde es necesario obtener soluciones rápidamente.

Aun con estas ventajas, la heurística presenta limitaciones inherentes a su propia estructura. En primer lugar, el uso de bloques predefinidos introduce una rigidez inicial que puede perjudicar el resultado final si los primeros bloques colocados no son compatibles con la geometría de la instancia. Esto se observa claramente en el contenedor más chico, donde bloques medianos dificultan la construcción posterior y llevan a reducciones en la función objetivo en un porcentaje no menor de instancias.

En segundo lugar, la heurística mantiene un carácter goloso y secuencial, ya que se decide primero qué bloque colocar y en qué orden, y después ajusta con bloques más chicos o cajas individuales. Esto implica que una mala decisión temprana es difícil de revertir, lo que explica por qué algunas estrategias (como la #5 en el segundo contenedor) muestran deterioros mayores o tiempos menos favorables.

Por otro lado, aunque la heurística es generalmente más rápida, no siempre acelera el tiempo de resolución. Existen casos puntuales (principalmente en instancias factibles mantenidas y en estrategias específicas) donde el tiempo promedio de la heurística supera al del modelo base. Estos episodios, aunque aislados, muestran que la estrategia no es universalmente superior y que su eficiencia depende de la interacción entre la forma del contenedor, las dimensiones de las cajas y la secuencia de bloques elegida.

Finalmente, la calidad de la solución también está condicionada por la propia idea de agrupar cajas: cuando la heurística reduce la cantidad de cajas, la pérdida promedio suele ser acotada, pero no deja de ser una pérdida estructural. En varios casos, la heurística sacrifica entre 3 y 6 cajas en promedio para obtener aceleraciones importantes en tiempo; aunque este trade-off puede ser conveniente, sigue representando una limitación intrínseca del enfoque.

14. Discusión

Con todos los resultados ya presentados para ambos contenedores, se propone interpretar los hallazgos en relación al marco teórico, destacando tanto los patrones esperados como aquellos comportamientos que se apartan de lo previsto. Dado que la heurística propuesta combina decisiones constructivas, secuenciales y dependientes de la geometría, su análisis requiere considerar múltiples dimensiones del problema. Por

este motivo, la discusión se organiza en cuatro ejes complementarios, que permiten abordar de manera estructurada: la relación con la literatura, la naturaleza del trade-off calidad–tiempo observado, la sensibilidad geométrica, el desempeño comparado de las estrategias, las ventajas prácticas del enfoque y sus principales limitaciones.

14.1 Relación entre los resultados obtenidos y el marco teórico

Los resultados obtenidos se alinean con los principales rasgos señalados en la literatura sobre el problema del Packing Tridimensional. Tal como anticipa la bibliografía, los modelos exactos basados en discretización muestran buenas soluciones en instancias simples, pero escalan con dificultad a medida que aumenta la complejidad geométrica; esto se refleja en los tiempos elevados, las soluciones parciales y los casos en los que el solver converge únicamente al limitar los cortes. A su vez, la restricción de estabilidad (documentada como un factor de incremento de la complejidad) se manifiesta en varios de los tiempos observados, ya que el solver tarda mucho más en en instancias donde hay que verificar muchas posibles áreas de apoyo y muchas de las instancias que caen en timeout o tardan una cantidad excesiva de tiempo son aquellas que generan configuraciones donde esta restricción es crítica.

En contraste, la literatura destaca el rol de las heurísticas constructivas como métodos capaces de reducir drásticamente los tiempos de búsqueda a costa de un posible deterioro moderado de la función objetivo. Los resultados en ambos contenedores para la enorme mayoría de las heurísticas muestran este comportamiento; cuando la heurística mejora o mantiene la solución, lo hace en tiempos bastante inferiores, y cuando la calidad disminuye, la pérdida resulta no tan acotada en comparación con la aceleración obtenida.

Finalmente, los trabajos previos también señalan la fuerte sensibilidad del problema a la geometría del contenedor. Esto se confirma empíricamente en el proyecto; los mismos modelados heurísticos muestran desempeños distintos en los dos escenarios analizados, así como dentro de ellos para cada heurística. Esto valida la idea de que los enfoques heurísticos deben interpretarse en función de las proporciones del espacio y no sólo de su propia lógica.

14.2 Comportamiento comparado del modelo exacto y las heurísticas

La comparación entre el modelo exacto y las heurísticas evidencia dos modos de funcionamiento claramente diferenciados, que resultan consistentes con lo señalado en la literatura para problemas tridimensionales de alta complejidad. El modelo exacto muestra una notable consistencia en términos de calidad (cuando logra la convergencia, produce soluciones óptimas), pero su desempeño temporal es extremadamente sensible a la discretización, a la restricción de estabilidad del 70% y a la geometría específica de cada instancia. Esto se refleja en tiempos que pueden variar desde fracciones de segundo hasta varios cientos, e incluso miles, de segundos. En este sentido, los resultados confirman las

limitaciones operativas ampliamente documentadas para formulaciones exactas de los problemas de 3D Packing de estas características.

Las heurísticas, en contraste, presentan un comportamiento temporal mucho más estable. Incluso considerando variaciones entre estrategias, sus rangos de tiempos tienden a mantenerse acotados y, en la mayoría de las instancias, significativamente por debajo del modelo base. Esto ocurre tanto cuando mejoran la solución como cuando simplemente la mantienen, y también cuando la empeoran; aun en esos casos, la pérdida promedio en cantidad de cajas resulta reducida en relación con la aceleración obtenida. En la mayoría de los escenarios donde el modelo exacto queda atrapado en tiempos elevados o en soluciones parciales, las heurísticas logran avances importantes, lo que pone de manifiesto la capacidad que tienen para explorar configuraciones viables evitando la complejidad combinatoria completa que enfrenta el modelo exacto.

En conjunto, ambos enfoques exponen de manera clara el trade-off que caracteriza a este tipo de problemas; el modelo exacto prioriza la calidad a expensas del tiempo, mientras que las heurísticas priorizan el tiempo a costa de aceptar una baja en la función objetivo. Los resultados obtenidos muestran que ningún método domina al otro en forma absoluta, pero sí revelan que la heurística constituye una alternativa especialmente valiosa en instancias donde la resolución exacta se vuelve ineficiente o inestable.

14.3 Sensibilidad geométrica y efectos del contenedor

Los resultados obtenidos muestran que el desempeño tanto del modelo exacto como de las heurísticas está fuertemente condicionado por la geometría del contenedor, un fenómeno recurrente en la bibliografía, pero que en este trabajo se evidencia de manera particularmente marcada. Entre ambos contenedores se observa un cambio significativo en la distribución de instancias que mejoran, empeoran o mantienen sus resultados, así como de sus performances en términos temporales. Esto ya que la cantidad de posiciones posibles aumenta, alterando la forma en que las cajas pueden apoyarse cumpliendo la restricción de estabilidad y reconfiguran la densidad factible de ser ocupada.

En el contenedor más pequeño, el espacio reducido limita las configuraciones posibles y, aunque el problema sigue siendo complejo, el modelo tiende a explorar un conjunto más acotado de alternativas. En el otro contenedor, en cambio, el aumento del espacio y de las combinaciones factibles incrementa la complejidad.

Estos resultados muestran que la sensibilidad geométrica no es un efecto secundario, sino un componente estructural de los problemas de esta índole. La variación en el tamaño de los contenedores, tiene un impacto comparable al que puede producir un cambio en la técnica, estrategia o parametrización aplicada, y debe ser muy tenida en cuenta al diseñar métodos para este tipo de problemas.

14.4 Ventajas, limitaciones e implicancias prácticas del enfoque aplicado

El estudio permite identificar un conjunto claro de fortalezas y limitaciones asociadas al enfoque empleado. Entre las ventajas más relevantes se destaca la posibilidad de obtener soluciones en tiempos considerablemente menores que los del modelo base, incluso en instancias donde este último presenta dificultades, cae en un timeout o devuelve soluciones subóptimas. La reducción del espacio de posibilidades que producen los métodos heurísticos contribuye a una matyor estabilidad temporal y a una mejor capacidad para resolver instancias complejas. De la misma forma, los resultados muestran que, en un número no menor de casos, se mantiene o incluso se mejora la calidad obtenida por el modelo base, lo que refuerza la utilidad de recurrir a técnicas no exactas cuando la escalabilidad es un factor clave.

No obstante, el enfoque también presenta limitaciones estructurales. Los métodos utilizados conservan un carácter constructivo y secuencial que los vuelve sensibles a decisiones tempranas, y en ciertos casos pueden producir soluciones de menor calidad que las del modelo exacto. Además, aunque los tiempos de resolución se reducen de manera marcada, existen instancias puntuales en las que los métodos aproximados no logran acelerar o incluso demandan más tiempo que la formulación exacta, lo que evidencia que su desempeño no es uniformemente superior.

En términos prácticos, el trabajo sugiere que este tipo de aproximaciones constituye una alternativa valiosa cuando se requieren tiempos de respuesta acotados, cuando el modelo exacto opera bajo restricciones severas o cuando la complejidad geométrica del caso vuelve impracticable una búsqueda exhaustiva. Sin embargo, en escenarios donde la calidad de la solución es prioritaria y los tiempos no son una restricción, el modelo exacto continúa siendo la referencia de mayor precisión. En conjunto, el enfoque adoptado demuestra ser útil y competitivo, pero su aplicación óptima depende del tipo de instancia, de la disponibilidad temporal y del perfil geométrico del contenedor.

15. Conclusiones & Recomendaciones

El trabajo permitió confirmar que el modelo exacto constituye una referencia sólida para evaluar soluciones, pero en la práctica se ve limitado por problemas de memoria y tiempos de ejecución crecientes a medida que aumenta la discretización y la cantidad de variables binarias. Frente a estas restricciones, las cuatro heurísticas desarrolladas demostraron ser herramientas efectivas para ampliar el rango de instancias resolubles y reducir significativamente los tiempos de cómputo, aun cuando esto implique sacrificar en algunos casos la garantía de optimalidad.

La heurística basada en relajación lineal mostró que es posible identificar posiciones prometedoras y reducir el tamaño del modelo de manera considerable. Esta estrategia permitió obtener aceleraciones muy altas en múltiples instancias, aunque con el riesgo asociado de eliminar variables relevantes y llegar a soluciones que son óptimas únicamente dentro del espacio filtrado. La falta de un patrón claro respecto

del valor ideal de ε sugiere que su desempeño está fuertemente condicionado por las características propias de cada instancia.

La heurística por capas logró simplificar de manera notable el problema tridimensional, transformándolo en varios problemas bidimensionales independientes. Este enfoque permitió tiempos de resolución excepcionalmente bajos y, en ciertos casos, incluso superar al modelo exacto cuando este no lograba cerrar una solución dentro del límite de tiempo. Sin embargo, forzar una única orientación y descartar el espacio residual entre capas generó pérdidas de calidad en la mayoría de las instancias, lo que evidencia una limitación estructural de este enfoque.

La extensión con relleno confirmó que combinar una construcción rápida por capas con una etapa adicional de optimización tridimensional mejora sistemáticamente la calidad, recuperando parte del espacio perdido y manteniendo tiempos razonables. Esta heurística se ubica en un punto intermedio equilibrado entre simplicidad y capacidad de exploración del espacio tridimensional.

Por último, la heurística basada en agrupamiento de cajas en bloques permitió reducir la complejidad combinatoria del problema al operar con unidades de mayor volumen. Su desempeño fue especialmente bueno en instancias amplias, aunque la efectividad se vió ligada a cuán bien la estructura de los bloques se alineó con las proporciones del contenedor. Esta dependencia introduce un nivel de sensibilidad que requiere seleccionar cuidadosamente los patrones de agrupamiento según el caso.

De forma transversal, los resultados obtenidos con todas las heurísticas reflejan un comportamiento consistente; las dimensiones relativas del contenedor y las cajas condicionan fuertemente la complejidad real de cada instancia y la eficacia de las estrategias aplicadas. Tanto la relajación lineal como la descomposición por capas, el llenado y el agrupamiento muestran mejoras o deterioros vinculados a cómo se alinean las orientaciones posibles, las proporciones de la caja y la capacidad efectiva de discretización. Este patrón, observado repetidamente a lo largo del análisis, permite concluir que la sensibilidad geométrica es una característica estructural del problema y un factor determinante para explicar por qué distintas heurísticas funcionan mejor o peor según la instancia.

Aunque el estudio trabajó sobre un conjunto acotado de contenedores y un esquema de discretización fijo, dicho conjunto resultó suficiente para obtener conclusiones sólidas dentro del alcance del proyecto. La variedad de instancias permitió observar patrones consistentes, contrastar enfoques y evaluar de manera razonable los trade-offs entre calidad de solución y tiempo de ejecución.

Parece relevante mencionar que varias ideas presentes en el material recibido al inicio del proyecto aparecen alineadas con lo que se observó en nuestras heurísticas. Por ejemplo, la propuesta de trabajar con subconjuntos reducidos de puntos del contenedor se vincula conceptualmente con el filtrado que realiza la relajación lineal, donde se busca concentrar el esfuerzo computacional en posiciones relevantes. De forma similar, la sugerencia de eliminar variables mediante criterios geométricos basados en

estabilidad coincide con los patrones detectados en nuestros experimentos; muchas posiciones no alcanzan el 70 % de soporte necesario, por lo que filtros más específicos podrían mejorar la eficiencia del modelo. También se destacan las ideas orientadas a fortalecer el modelo exacto mediante desigualdades válidas, que resultan coherentes con la influencia decisiva que mostró la estructura geométrica en la calidad del espacio de búsqueda.

A partir de estos elementos, las líneas futuras más naturales incluyen explorar versiones adaptativas de los métodos implementados, incorporar discretizaciones que se ajusten dinámicamente al avance de la construcción y aplicar filtros geométricos más precisos para depurar el espacio de búsqueda. Estas direcciones combinan lo sugerido por el material inicial con lo observado empíricamente durante el desarrollo del trabajo y representan una extensión razonable para continuar mejorando el rendimiento y la escalabilidad de los modelos de 3D Packing.

16. Referencias bibliográficas

- **Bortfeldt, A., & Wäscher, G.** (2013). Constraints in container loading – A state-of-the-art review. European Journal of Operational Research, 229(1), 1-20.
- **Garey, M. R., & Johnson, D. S.** (1979). Computers and intractability: A guide to the theory of NP-completeness. W. H. Freeman.
- **Gosling, J., Joy, B., Steele, G., Bracha, G., & Buckley, A.** (2014). The Java language specification, Java SE 8 edition. Addison-Wesley Professional.
- **IBM**. (2021). IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/products/ilog-cplex-optimization-studio
- **Scheithauer, G.** (1999). LP-based bounds for the container and multi-container loading problem. International Transactions in Operational Research, 6(2), 199-213.
- **Talbi, E. G.** (2009). Metaheuristics: from design to implementation. John Wiley & Sons.
- **Wäscher, G., Haußner, H., & Schumann, H.** (2007). An improved typology of cutting and packing problems. European Journal of Operational Research, 183(3), 1109-1130.
- **Wikimedia Commons**. (2024). P np np-complete np-hard.svg. Recuperado de https://commons.wikimedia.org/wiki/File:P_np_np-complete_np-hard.svg
- **Wolsey, L. A.** (1998). Integer programming. Wiley-Interscience

17. Apéndices

Apéndice A: Resultados del código base

Para el contenedor 260 x 240 x 110:

Caja	Tiempo	Estado	Valor
(30, 30, 85)	1,039	Optimal	72
(30, 30, 84)	1,027	Optimal	72
(40, 35, 75)	97,489	Optimal	63
(40, 43, 84)	28,237	Optimal	36
(40, 40, 75)	0,318	Optimal	36
(40, 45, 84)	42,440	Optimal	36
(40, 40, 76)	0,309	Optimal	36
(40, 37, 75)	253,392	Optimal	42
(41, 37, 75)	396,472	Optimal	42
(41, 24, 113)	619,908	Feasible	53
(45, 35, 61)	646,780	Feasible	60
(45, 32, 84)	611,142	Feasible	43
(45, 30, 84)	603,468	Feasible	54
(50, 45, 109)	2,809	Optimal	25
(50, 50, 91)	0,188	Optimal	25
(55, 40, 80)	1,360	Optimal	36
(60, 20, 141)	7,320	Optimal	29
(62, 39, 92)	6,279	Optimal	25
(64, 39, 93)	8,704	Optimal	25
(69, 40, 95)	3,053	Optimal	24
(125, 19, 118)	1,137	Optimal	20
(130, 55, 75)	0,182	Optimal	12
(130, 45, 62)	0,772	Optimal	17
(135, 29, 47)	604,216	Feasible	32
(135, 29, 46)	604,582	Feasible	32
(138, 30, 47)	20,494	Optimal	30
(140, 49, 52)	0,838	Optimal	16
(160, 33, 46)	4,603	Optimal	22
(164, 67, 73)	0,142	Optimal	4
(170, 85, 45)	0,187	Optimal	7
(171, 35, 70)	0,180	Optimal	12

(185, 55, 55)	0,148	Optimal	10
(185, 55, 56)	0,260	Optimal	10
(190, 175, 45)	0,123	Optimal	2
(195, 36, 130)	0,166	Optimal	6

Para el contenedor 400 x 240 x 130:

Caja	Tiempo_resolucion	Estado	Obj_value
(30, 30, 85)	20,655	Optimal	149
(30, 30, 84)	28,492	Optimal	149
(40, 35, 75)	618,844	Feasible	99
(40, 43, 84)	718,756	Feasible	74
(40, 40, 75)	12,211	Optimal	90
(40, 45, 84)	692,304	Feasible	74
(40, 40, 76)	4,112	Optimal	90
(40, 37, 75)	966,653	Feasible	0
(41, 37, 75)	1.094,801	Feasible	0
(41, 24, 113)	700,830	Feasible	89
(45, 35, 61)	1.473,243	Feasible	0
(45, 32, 84)	779,438	Feasible	90
(45, 30, 84)	632,769	Feasible	95
(50, 45, 109)	86,058	Optimal	40
(50, 50, 91)	0,285	Optimal	40
(55, 40, 80)	43,825	Optimal	63
(60, 20, 141)	59,228	Optimal	60
(62, 39, 92)	301,127	Optimal	50
(64, 39, 93)	328,600	Optimal	50
(69, 40, 95)	39,617	Optimal	42
(125, 19, 118)	78,761	Optimal	42
(130, 55, 75)	0,996	Optimal	22
(130, 45, 62)	10,533	Optimal	32
(135, 29, 47)	647,342	Feasible	49
(135, 29, 46)	667,635	Feasible	51
(138, 30, 47)	666,952	Feasible	49
(140, 49, 52)	5,220	Optimal	26
(160, 33, 46)	616,901	Feasible	41
(164, 67, 73)	0,153	Optimal	7
(170, 85, 45)	0,446	Optimal	15

(171, 35, 70)	0,243	Optimal	19
(185, 55, 55)	0,136	Optimal	18
(185, 55, 56)	1,274	Optimal	18
(190, 175, 45)	0,154	Optimal	4
(195, 36, 130)	0,352	Optimal	13

Apéndice B: Resultados de la relajación lineal

Para el contenedor 260 x 240 x 110:

Valor epsilon	**0,0001**	**0,001**	**0,01**	**0,1**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja	**Variables_eliminadas**				**Tiempo_resolucion**			
(30, 30, 85)	2262	2262	2262	2262	0,6559989	0,0522373	0,0437243	0,0350134
(30, 30, 84)	2262	2262	2262	2262	0,5966219	0,0528104	0,0449974	0,0348732
(40, 35, 75)	7831	7848	7972	8567	121,5139995	28,0780921	10,1994153	0,142382
(40, 43, 84)	7116	7116	7116	7116	36,8496241	0,0731941	0,0307341	0,032143
(40, 40, 75)	1116	1116	1116	1116	0,1280131	0,0585476	0,0365087	0,0468793
(40, 45, 84)	6734	6734	6734	6734	52,0530563	0,0460039	0,043086	0,0378319
(40, 40, 76)	1116	1116	1116	1116	0,1210835	0,0523015	0,0483636	0,0441857
(40, 37, 75)	11515	11471	11471	11568	294,129933	0,2280973	0,148711	0,0999954
(41, 37, 75)	11184	11304	11304	11304	499,2157717	0,0737595	0,0779271	0,0697329
(41, 24, 113)	11198	11200	11230	11248	600,597043	0,0914003	0,0710435	0,0727344
(45, 35, 61)								
(45, 32, 84)		13205	13205	13205		0,1206742	0,1196343	0,110772
(45, 30, 84)	8011	8011	8011	8013	600,6058056	0,0598594	0,0644445	0,0678771
(50, 45, 109)	2463	2483	2493	2583	2,524947	0,0920395	0,0877754	0,0525142
(50, 50, 91)	449	449	449	449	0,0266583	0,0412896	0,0451817	0,0425819
(55, 40, 80)	2231	2240	2240	2240	0,8388916	0,08229	0,0588866	0,0529362
(60, 20, 141)	1116	1116	1144	1217	8,398738	0,591592	0,1778044	0,0536592
(62, 39, 92)	2436	2440	2462	2651	6,7827402	0,6784552	0,6257984	0,0668261
(64, 39, 93)	2429	2437	2455	2650	10,0810575	0,698429	0,7409684	0,0483926
(69, 40, 95)	1937	1932	1932	2087	3,1095393	0,3608798	0,2969954	0,0552587
(125, 19, 118)	765	765	765	765	0,2456336	0,0476606	0,036753	0,0338646
(130, 55, 75)	198	198	198	198	0,0219983	0,0490987	0,0397358	0,0384882
(130, 45, 62)	783	783	783	783	0,3471346	0,0752299	0,0513885	0,0545114
(135, 29, 47)	4413	4413	4413	4437	600,325085	0,0632878	0,0615955	0,0582829

					6			
(135, 29, 46)	4494	4494	4494	4494	600,4084018	0,0753171	0,0640847	0,0604996
(138, 30, 47)	3550	3664	3664	3665	22,0456881	0,0450405	0,0473676	0,0496934
(140, 49, 52)	830	830	830	830	0,4059681	0,0534498	0,0409345	0,0397545
(160, 33, 46)	1906	1906	1906	1906	3,6160248	0,0382743	0,0340622	0,0310748
(164, 67, 73)	80	80	80	80	0,0122079	0,0394847	0,0312251	0,0303784
(170, 85, 45)	129	129	129	129	0,0245231	0,0363385	0,0422857	0,0332188
(171, 35, 70)	143	143	143	143	0,0270582	0,0584444	0,0539004	0,0629967
(185, 55, 55)	70	70	70	70	0,0109773	0,0368109	0,0327838	0,0338514
(185, 55, 56)	149	149	149	149	0,0315716	0,0682376	0,0405378	0,0771065
(190, 175, 45)	14	14	14	14	0,0088066	0,0518262	0,0304465	0,0324399
(195, 36, 130)	48	48	48	48	0,0125911	0,0647715	0,0396278	0,03916

Valor epsilon	**0,0001**	**0,001**	**0,01**	**0,1**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja	**Estado**				**Obj_value**			
(30, 30, 85)	Optimal	Optimal	Optimal	Optimal	72	72	72	72
(30, 30, 84)	Optimal	Optimal	Optimal	Optimal	72	72	72	72
(40, 35, 75)	Optimal	Optimal	Optimal	Optimal	63	63	63	56
(40, 43, 84)	Optimal	Optimal	Optimal	Optimal	36	36	36	36
(40, 40, 75)	Optimal	Optimal	Optimal	Optimal	36	36	36	36
(40, 45, 84)	Optimal	Optimal	Optimal	Optimal	36	36	36	36
(40, 40, 76)	Optimal	Optimal	Optimal	Optimal	36	36	36	36
(40, 37, 75)	Optimal	Optimal	Optimal	Optimal	42	42	42	42
(41, 37, 75)	Optimal	Optimal	Optimal	Optimal	42	42	42	42
(41, 24, 113)	Feasible	Optimal	Optimal	Optimal	53	56	56	56
(45, 35, 61)	Timeout	Timeout	Timeout	Timeout				
(45, 32, 84)	Timeout	Optimal	Optimal	Optimal		48	48	48
(45, 30, 84)	Feasible	Optimal	Optimal	Optimal	54	54	54	54
(50, 45, 109)	Optimal	Optimal	Optimal	Optimal	25	25	25	25

(50, 50, 91)	Optimal	Optimal	Optimal	Optimal	25	25	25	25
(55, 40, 80)	Optimal	Optimal	Optimal	Optimal	36	35	35	35
(60, 20, 141)	Optimal	Optimal	Optimal	Optimal	29	26	26	24
(62, 39, 92)	Optimal	Optimal	Optimal	Optimal	25	25	25	25
(64, 39, 93)	Optimal	Optimal	Optimal	Optimal	25	25	25	24
(69, 40, 95)	Optimal	Optimal	Optimal	Optimal	24	24	24	24
(125, 19, 118)	Optimal	Optimal	Optimal	Optimal	20	20	20	20
(130, 55, 75)	Optimal	Optimal	Optimal	Optimal	12	12	12	12
(130, 45, 62)	Optimal	Optimal	Optimal	Optimal	17	17	17	17
(135, 29, 47)	Feasible	Optimal	Optimal	Optimal	32	32	32	29
(135, 29, 46)	Feasible	Optimal	Optimal	Optimal	32	28	28	28
(138, 30, 47)	Optimal	Optimal	Optimal	Optimal	30	28	28	28
(140, 49, 52)	Optimal	Optimal	Optimal	Optimal	16	16	16	16
(160, 33, 46)	Optimal	Optimal	Optimal	Optimal	22	22	22	22
(164, 67, 73)	Optimal	Optimal	Optimal	Optimal	4	4	4	4
(170, 85, 45)	Optimal	Optimal	Optimal	Optimal	7	7	7	7
(171, 35, 70)	Optimal	Optimal	Optimal	Optimal	12	12	12	12
(185, 55, 55)	Optimal	Optimal	Optimal	Optimal	10	10	10	10
(185, 55, 56)	Optimal	Optimal	Optimal	Optimal	10	10	10	10
(190, 175, 45)	Optimal	Optimal	Optimal	Optimal	2	2	2	2
(195, 36, 130)	Optimal	Optimal	Optimal	Optimal	6	6	6	6

Para el contenedor 400 x 240 x 130:

Valor epsilon	**0,0001**	**0,001**	**0,01**	**0,1**	**Base**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja	**Variables_eliminadas**				**Tiempo_resolucion**				
(30, 30, 85)	9138	9138	9138	9138	20,655	0,1591462	0,177582501	0,1839196	0,1887528
(30, 30, 84)	9054	9054	9054	9146	28,492	0,2590673	0,236257100	0,2643413	0,0884623

(40, 35, 75)					618,844				
(40, 43, 84)					718,756				
(40, 40, 75)	5384	5384	5397	5695	12,211	5,8740979	5,125554099	4,5725391	0,1716770
(40, 45, 84)					692,304				
(40, 40, 76)	5388	5388	5393	5683	4,112	4,5124347	4,764348101	7,4663151	0,2478545
(40, 37, 75)					966,653				
(41, 37, 75)					1.094,801				
(41, 24, 113)					700,830				
(45, 35, 61)					1.473,243				
(45, 32, 84)					779,438				
(45, 30, 84)					632,769				
(50, 45, 109)	8188	8188	8211	8586	86,058	0,6330347	0,710241601	0,9152606	0,1199239
(50, 50, 91)	1088	1088	1088	1088	0,285	0,0440269	0,037386000	0,0462027	0,0445415
(55, 40, 80)	6598	6598	6613	7037	43,825	1,6352815	1,579744600	1,3096791	0,0646686
(60, 20, 141)	4140	4140	4140	4145	59,228	0,0476600	0,048851200	0,0537825	0,0450168
(62, 39, 92)	11268	11268	11268	11268	301,127	0,0425650	0,037719700	0,0369027	0,0382389
(64, 39, 93)	11469	11469	11469	11469	328,600	0,0657130	0,070913899	0,0629554	0,0571235
(69, 40, 95)	7965	7965	7965	7965	39,617	0,0404693	0,036017600	0,0355197	0,0410090
(125, 19, 118)	6612	6612	6612	6612	78,761	0,0392923	0,042945900	0,0383747	0,0458638
(130, 55, 75)	932	932	951	1077	0,996	0,1224382	0,108846500	0,2107869	0,0562009
(130, 45, 62)	4218	4218	4220	4333	10,533	0,1423681	0,157374500	0,1241434	0,0525038
(135, 29, 47)					647,342				

(135, 29, 46)					667,635				
(138, 30, 47)					666,952				
(140, 49, 52)	3001	3001	3001	3001	5,220	0,0508288	0,033250100	0,0302582	0,0313530
(160, 33, 46)					616,901				
(164, 67, 73)	290	290	290	290	0,153	0,0413023	0,043657700	0,0442379	0,0470868
(170, 85, 45)	612	612	613	629	0,446	0,0811742	0,092184100	0,1085709	0,0677767
(171, 35, 70)	396	396	396	396	0,243	0,0624479	0,060071300	0,0594351	0,0758506
(185, 55, 55)	162	162	162	162	0	0,0536698	0,047365300	0,0482896	0,0454893
(185, 55, 56)	1065	1065	1065	1065	1,274	0,0569016	0,053014300	0,0493339	0,0558518
(190, 175, 45)	60	60	60	60	0,154	0,0370073	0,030344500	0,0329978	0,0330411
(195, 36, 130)	359	359	359	359	0,352	0,0380106	0,036929800	0,0375716	0,0341008

Valor epsilon	**Base**	**0,0001**	**0,001**	**0,01**	**0,1**	**Base**	**0,0001**	**0,001**	**0,01**	**0,1**
Caja		**Estado**					**Obj_value**			
(30, 30, 85)	Optimal	Optimal	Optimal	Optimal	Optimal	149	146	146	146	146
(30, 30, 84)	Optimal	Optimal	Optimal	Optimal	Optimal	149	148	148	148	148
(40, 35, 75)	Feasible	Timeout	Timeout	Timeout	Timeout	99				
(40, 43, 84)	Feasible	Timeout	Timeout	Timeout	Timeout	74				
(40, 40, 75)	Optimal	Optimal	Optimal	Optimal	Optimal	90	89	89	89	87
(40, 45, 84)	Feasible	Timeout	Timeout	Timeout	Timeout	74				
(40, 40, 76)	Optimal	Optimal	Optimal	Optimal	Optimal	90	89	89	89	86
(40, 37, 75)	Timeout	Timeout	Timeout	Timeout	Timeout	0				
(41, 37, 75)	Timeout	Timeout	Timeout	Timeout	Timeout	0				
(41, 24, 113)	Feasible	Timeout	Timeout	Timeout	Timeout	89				

(45, 35, 61)	Timeout	Timeout	Timeout	Timeout	Timeout	0				
(45, 32, 84)	Feasible	Timeout	Timeout	Timeout	Timeout	90				
(45, 30, 84)	Feasible	Timeout	Timeout	Timeout	Timeout	95				
(50, 45, 109)	Optimal	Optimal	Optimal	Optimal	Optimal	40	40	40	40	38
(50, 50, 91)	Optimal	Optimal	Optimal	Optimal	Optimal	40	40	40	40	40
(55, 40, 80)	Optimal	Optimal	Optimal	Optimal	Optimal	63	63	63	63	63
(60, 20, 141)	Optimal	Optimal	Optimal	Optimal	Optimal	60	58	58	58	57
(62, 39, 92)	Optimal	Optimal	Optimal	Optimal	Optimal	50	50	50	50	50
(64, 39, 93)	Optimal	Optimal	Optimal	Optimal	Optimal	50	50	50	50	50
(69, 40, 95)	Optimal	Optimal	Optimal	Optimal	Optimal	42	42	42	42	42
(125, 19, 118)	Optimal	Optimal	Optimal	Optimal	Optimal	42	42	42	42	42
(130, 55, 75)	Optimal	Optimal	Optimal	Optimal	Optimal	22	22	22	22	22
(130, 45, 62)	Optimal	Optimal	Optimal	Optimal	Optimal	32	32	32	32	32
(135, 29, 47)	Feasible	Timeout	Timeout	Timeout	Timeout	49				
(135, 29, 46)	Feasible	Timeout	Timeout	Timeout	Timeout	51				
(138, 30, 47)	Feasible	Timeout	Timeout	Timeout	Timeout	49				
(140, 49, 52)	Optimal	Optimal	Optimal	Optimal	Optimal	26	26	26	26	26
(160, 33, 46)	Feasible	Timeout	Timeout	Timeout	Timeout	41				
(164, 67, 73)	Optimal	Optimal	Optimal	Optimal	Optimal	7	7	7	7	7
(170, 85, 45)	Optimal	Optimal	Optimal	Optimal	Optimal	15	12	12	12	12
(171, 35, 70)	Optimal	Optimal	Optimal	Optimal	Optimal	19	19	19	19	19
(185, 55, 55)	Optimal	Optimal	Optimal	Optimal	Optimal	18	18	18	18	18
(185, 55, 56)	Optimal	Optimal	Optimal	Optimal	Optimal	18	18	18	18	18
(190, 175, 45)	Optimal	Optimal	Optimal	Optimal	Optimal	4	4	4	4	4
(195, 36, 130)	Optimal	Optimal	Optimal	Optimal	Optimal	13	13	13	13	13

Apéndice C: Resultados Heuristica por capas

Para el contenedor 260 x 240 x 110:

Caja	Mejor orientación	Cajas por capa	Total cajas	Tiempo total
(30,30,85)	HW	24, 24, 24	72	0.29
(30,30,84)	HW	24, 24, 24	72	0.29
(40,35,75)	HL	18, 18, 18	54	0.39
(40,43,84)	WL	36	36	0.68
(40,40,75)	LW	36	36	0.27
(40,45,84)	HL	18, 18	36	0.65
(40,40,76)	LW	36	36	0.25
(40,37,75)	WL	42	42	1.21
(41,37,75)	WH	21, 21	42	1.2
(41,24,113)	LH	12, 12, 12, 12	48	0.71
(45,35,61)	HL	20, 20, 20	60	1.95
(45,30,84)	HW	24, 24	48	0.45
(45,32,84)	HL	15, 15, 15	45	0.88
(45,30,84)	HW	24, 24	48	0.47
(50,45,109)	LW	25	25	0.36
(50,50,91)	LW	20	20	0.18
(55,40,80)	HW	18, 18	36	0.28
(60,20,141)	HL	4, 4, 4, 4, 4	20	0.21
(62,39,92)	LW	24	24	0.38
(64,39,93)	LW	24	24	0.6
(69,40,95)	LW	18	18	0.46
(125,19,118)	LH	4, 4, 4, 4, 4	20	0.34
(130,55,75)	LH	6, 6	12	0.19
(130,45,62)	LH	6, 6	12	0.3
(135,29,47)	LW	8, 8	16	0.66
(135,29,46)	LW	8, 8	16	0.66
(138,30,47)	LW	8, 8	16	0.52
(140,49,52)	WL	5, 5	10	0.24
(160,33,46)	HL	5, 5, 5	15	0.43
(164,67,73)	LW	3	3	0.18
(170,85,45)	WL	3, 3	6	0.16
(171,35,70)	HL	3, 3, 3	9	0.16

(185,55,55)	LW	4, 4	8	0.16
(185,55,56)	HL	4, 4	8	0.18
(190,175,45)	LW	1, 1	2	0.16
(195,36,130)	HL	2, 2, 2	6	0.16

Para el contenedor 400 x 240 x 130:

Caja	Mejor orientación	Cajas_por_capa	Total cajas	Tiempo total
(30,30,85)	HW	32, 32, 32, 32	128	0,45
(30,30,84)	HW	32, 32, 32, 32	128	0,45
(40,35,75)	WH	33, 33, 33	99	0,91
(40,43,84)	HL	24, 24, 24	72	2,71
(40,40,75)	HW	30, 30, 30	90	0,52
(40,45,84)	HW	20, 20, 20	60	4,14
(40,40,76)	HW	30, 30, 30	90	0,45
(40,37,75)	HW	30, 30, 30	90	12,3
(41,37,75)	HW	30, 30, 30	90	9,78
(41,24,113)	WH	32, 32, 32	96	1,91
(45,35,61)	WL	55, 55	110	16,1
(45,30,84)	HL	20, 20, 20, 20	80	1,18
(45,32,84)	HL	20, 20, 20, 20	80	3,68
(45,30,84)	HL	20, 20, 20, 20	80	1,21
(50,45,109)	LW	40	40	1,01
(50,50,91)	LW	32	32	0,29
(55,40,80)	LH	21, 21, 21	63	0,48
(60,20,141)	HW	24, 24	48	0,35
(62,39,92)	HW	24, 24	48	1,17
(64,39,93)	HW	24, 24	48	1,29
(69,40,95)	HL	12, 12, 12	36	0,95
(125,19,118)	WH	42	42	1
(130,55,75)	WH	21	21	0,27
(130,45,62)	LW	15, 15	30	0,59
(135,29,47)	LH	10, 10, 10, 10	40	1,78
(135,29,46)	LH	10, 10, 10, 10	40	1,81
(138,30,47)	LH	10, 10, 10, 10	40	1,54
(140,49,52)	LW	8, 8	16	0,43
(160,33,46)	LH	10, 10, 10	30	1,43

(164,67,73)	LW	6	6	0,26
(170,85,45)	LH	10	10	0,24
(171,35,70)	LH	6, 6, 6	18	0,21
(185,55,55)	LW	8, 8	16	0,16
(185,55,56)	LW	8, 8	16	0,43
(190,175,45)	LW	2, 2	4	0,19
(195,36,130)	LW	12	12	0,25

Apéndice C: Resultados heuristica por capas con relleno 3D

Para el contenedor 260 x 240 x 110:

Caja	Mejor orientación	Capas usadas	Cajas por capa	Total sin relleno	Cajas relleno 3D	Total cajas	Tiempo total
(30,30,85)	HW	3	24	72	0	72	0,02
(30,30,84)	HW	3	24	72	0	72	0,02
(40,35,75)	HL	3	18	54	24	78	0,13
(40,43,84)	WL	1	36	36	0	36	0,11
(40,40,75)	LW	1	36	36	0	36	0,03
(40,45,84)	HL	2	18	36	0	36	0,07
(40,40,76)	LW	1	36	36	0	36	0,02
(40,37,75)	WL	1	42	42	0	42	0,17
(41,37,75)	WH	2	21	42	0	42	0,12
(41,24,113)	LH	4	12	48	12	60	0,35
(45,35,61)	HL	3	20	60	0	60	0,24
(45,30,84)	HW	2	24	48	15	63	0,13
(45,32,84)	HL	3	15	45	15	60	0,65
(45,30,84)	HW	2	24	48	15	63	0,13
(50,45,109)	LW	1	25	25	0	25	0,05
(50,50,91)	LW	1	20	20	0	20	0,01
(55,40,80)	HW	2	18	36	0	36	0,02
(60,20,141)	HL	5	4	20	7	27	0,21
(62,39,92)	LW	1	24	24	0	24	0,05
(64,39,93)	LW	1	24	24	0	24	0,05
(69,40,95)	LW	1	18	18	2	20	0,04
(125,19,118)	LH	5	4	20	0	20	0,03
(130,55,75)	LH	2	6	12	0	12	0,01
(130,45,62)	LH	2	6	12	2	14	0,03
(135,29,47)	LW	2	8	16	9	25	0,17
(135,29,46)	LW	2	8	16	11	27	0,31
(138,30,47)	LW	2	8	16	6	22	0,07
(140,49,52)	WL	2	5	10	2	12	0,02
(160,33,46)	HL	3	5	15	0	15	0,03
(164,67,73)	LW	1	3	3	1	4	0,01
(170,85,45)	WL	2	3	6	0	6	0,01

(171,35,70)	HL	3	3	9	1	10	0,01
(185,55,55)	LW	2	4	8	2	10	0,01
(185,55,56)	HL	2	4	8	0	8	0,01
(190,175,45)	LW	2	1	2	0	2	0,01
(195,36,130)	HL	3	2	6	0	6	0,01

Para el contenedor 400 x 240 x 130:

Caja	Mejor orientación	Capas usadas	Cajas por capa	Total sin relleno	Cajas relleno 3D	Total cajas	Tiempo total
(30,30,85)	HW	4	32	128	20	148	0,08
(30,30,84)	HW	4	32	128	20	148	0,09
(40,35,75)	WH	3	33	99	33	132	0,59
(40,43,84)	HL	3	24	72	31	103	8,77
(40,40,75)	HW	3	30	90	0	90	0,03
(40,45,84)	HW	3	20	60	28	88	39,66
(40,40,76)	HW	3	30	90	0	90	0,04
(40,37,75)	HW	3	30	90	30	120	54,09
(41,37,75)	HW	3	30	90	30	120	44,34
(41,24,113)	WH	3	32	96	33	129	60,83
(45,35,61)	WL	2	55	110	0	110	1,52
(45,30,84)	HL	4	20	80	32	112	1,98
(45,32,84)	HL	4	20	80	32	112	11,07
(45,30,84)	HL	4	20	80	32	112	1,55
(50,45,109)	LW	1	40	40	0	40	0,1
(50,50,91)	LW	1	32	32	0	32	0,02
(55,40,80)	LH	3	21	63	0	63	0,04
(60,20,141)	HW	2	24	48	26	74	1,07
(62,39,92)	HW	2	24	48	0	48	0,15
(64,39,93)	HW	2	24	48	0	48	0,15
(69,40,95)	HL	3	12	36	0	36	0,11
(125,19,118)	WH	1	42	42	0	42	0,07
(130,55,75)	WH	1	21	21	0	21	0,02
(130,45,62)	LW	2	15	30	0	30	0,06
(135,29,47)	LH	4	10	40	20	60	60,45
(135,29,46)	LH	4	10	40	20	60	60,43
(138,30,47)	LH	4	10	40	20	60	60,38

(140,49,52)	LW	2	8	16	4	20	0,12
(160,33,46)	LH	3	10	30	20	50	5,82
(164,67,73)	LW	1	6	6	1	7	0,02
(170,85,45)	LH	1	10	10	5	15	0,05
(171,35,70)	LH	3	6	18	1	19	0,01
(185,55,55)	LW	2	8	16	0	16	0,01
(185,55,56)	LW	2	8	16	0	16	0,04
(190,175,45)	LW	2	2	4	0	4	0,01
(195,36,130)	LW	1	12	12	0	12	0,02

Apéndice D: Resultados heuristica de agrupamiento de cajas en bloques

Para el contenedor 260x240x110:

- Estrategia #1:

Caja	Total Cajas	Total Tiempo	Estado Respuesta
(30, 30, 85)	72	0.638	Optima
(30, 30, 84)	72	0.419	Optima
(40, 35, 75)	60	3.825	Factible
(40, 43, 84)	36	12.249	Optima
(40, 40, 75)	36	0.084	Optima
(40, 45, 84)	36	7.606	Optima
(40, 40, 76)	36	0.085	Optima
(40, 37, 75)	36	18.999	Factible
(41, 37, 75)	36	19.6	Factible
(41, 24, 113)	50	15.682	Factible
(45, 35, 61)	60	61.195	Factible
(45, 30, 84)	44	4.526	Factible
(45, 32, 84)	40	20.653	Factible
(50, 45, 109)	25	2.01	Optima
(50, 50, 91)	23	0.034	Factible
(55, 40, 80)	26	0.246	Factible
(60, 20, 141)	25	0.306	Factible
(62, 39, 92)	21	1.642	Factible
(64, 39, 93)	21	1.678	Factible
(69, 40, 95)	21	1.169	Factible
(125, 19, 118)	20	1.923	Optima
(130, 55, 75)	12	0.019	Optima
(130, 45, 62)	14	0.358	Factible
(135, 29, 47)	24	9.262	Factible
(135, 29, 46)	24	10.16	Factible

(138, 30, 47)	24	7.439	Factible
(140, 49, 52)	16	0.565	Optima
(160, 33, 46)	20	3.858	Factible
(164, 67, 73)	4	0.027	Optima
(170, 85, 45)	7	0.037	Optima
(171, 35, 70)	11	0.024	Factible
(185, 55, 55)	10	0.009	Optima
(185, 55, 56)	9	0.061	Factible
(190, 175, 45)	2	0.013	Optima
(195, 36, 130)	6	0.044	Optima

- Estrategia #2:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	64	0.239	Factible
(30, 30, 84)	64	0.262	Factible
(40, 35, 75)	49	2.949	Factible
(40, 43, 84)	36	9.176	Optima
(40, 40, 75)	36	0.07	Optima
(40, 45, 84)	36	6.805	Optima
(40, 40, 76)	36	0.068	Optima
(40, 37, 75)	40	19.084	Factible
(41, 37, 75)	38	19.413	Factible
(41, 24, 113)	52	14.965	Factible
(45, 35, 61)	52	57.066	Factible
(45, 30, 84)	48	3.72	Factible
(45, 32, 84)	44	19.847	Mejor
(50, 45, 109)	25	1.703	Optima
(50, 50, 91)	22	0.027	Factible
(55, 40, 80)	36	0.216	Optima
(60, 20, 141)	27	0.396	Factible

(62, 39, 92)	24	1.245	Factible
(64, 39, 93)	24	1.289	Factible
(69, 40, 95)	22	0.914	Factible
(125, 19, 118)	20	1.362	Optima
(130, 55, 75)	12	0.013	Optima
(130, 45, 62)	14	0.272	Factible
(135, 29, 47)	28	8.915	Factible
(135, 29, 46)	28	9.29	Factible
(138, 30, 47)	28	6.442	Factible
(140, 49, 52)	16	0.478	Optima
(160, 33, 46)	20	3.428	Factible
(164, 67, 73)	4	0.02	Optima
(170, 85, 45)	7	0.032	Optima
(171, 35, 70)	10	0.021	Factible
(185, 55, 55)	10	0.007	Optima
(185, 55, 56)	9	0.049	Factible
(190, 175, 45)	2	0.008	Optima
(195, 36, 130)	6	0.029	Optima

- Estrategia #3:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	72	1.832	Optima
(30, 30, 84)	72	0.527	Optima
(40, 35, 75)	56	5.953	Factible
(40, 43, 84)	34	21.819	Factible
(40, 40, 75)	36	0.228	Optima
(40, 45, 84)	34	21.084	Factible
(40, 40, 76)	36	0.222	Optima
(40, 37, 75)	36	34.969	Factible

(41, 37, 75)	36	43.084	Factible
(41, 24, 113)	48	36.057	Factible
(45, 35, 61)	60	102.66	Factible
(45, 30, 84)	46	6.845	Factible
(45, 32, 84)	46	33.028	Mejor
(50, 45, 109)	20	3.435	Factible
(50, 50, 91)	20	0.048	Factible
(55, 40, 80)	28	0.421	Factible
(60, 20, 141)	24	0.46	Factible
(62, 39, 92)	19	2.898	Factible
(64, 39, 93)	19	2.756	Factible
(69, 40, 95)	15	2.204	Factible
(125, 19, 118)	20	4.206	Optima
(130, 55, 75)	12	0.06	Optima
(130, 45, 62)	14	0.97	Factible
(135, 29, 47)	32	34.446	Factible
(135, 29, 46)	32	37.01	Factible
(138, 30, 47)	29	25.033	Factible
(140, 49, 52)	16	0.979	Optima
(160, 33, 46)	19	11.45	Factible
(164, 67, 73)	4	0.043	Optima
(170, 85, 45)	6	0.063	Factible
(171, 35, 70)	11	0.04	Factible
(185, 55, 55)	10	0.041	Optima
(185, 55, 56)	10	0.117	Optima
(190, 175, 45)	2	0.023	Optima
(195, 36, 130)	6	0.072	Optima

- Estrategia #4:

Caja	Total Cajas	Total Tiempo	Estado Respuesta
(30, 30, 85)	72	0.647	Optima
(30, 30, 84)	72	0.475	Optima
(40, 35, 75)	45	4.047	Factible
(40, 43, 84)	34	13.181	Factible
(40, 40, 75)	36	0.182	Optima
(40, 45, 84)	34	17.724	Factible
(40, 40, 76)	36	0.176	Optima
(40, 37, 75)	36	25.439	Factible
(41, 37, 75)	36	27.378	Factible
(41, 24, 113)	52	38.654	Factible
(45, 35, 61)	48	95.393	Optima
(45, 30, 84)	47	5.824	Factible
(45, 32, 84)	42	30.829	Factible
(50, 45, 109)	21	2.858	Factible
(50, 50, 91)	20	0.038	Factible
(55, 40, 80)	36	0.372	Optima
(60, 20, 141)	24	0.379	Factible
(62, 39, 92)	20	2.186	Factible
(64, 39, 93)	20	2.166	Factible
(69, 40, 95)	17	2.373	Factible
(125, 19, 118)	20	4.385	Optima
(130, 55, 75)	12	0.039	Optima
(130, 45, 62)	14	0.842	Factible
(135, 29, 47)	32	25.712	Factible
(135, 29, 46)	32	28.595	Factible
(138, 30, 47)	29	19.426	Factible
(140, 49, 52)	16	0.862	Optima
(160, 33, 46)	19	8.25	Factible
(164, 67, 73)	4	0.033	Optima

(170, 85, 45)	6	0.05	Factible
(171, 35, 70)	11	0.029	Factible
(185, 55, 55)	10	0.027	Optima
(185, 55, 56)	10	0.094	Optima
(190, 175, 45)	2	0.017	Optima
(195, 36, 130)	6	0.057	Optima

- Estrategia #5:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	72	1.681	Optima
(30, 30, 84)	72	1.454	Optima
(40, 35, 75)	60	45.423	Factible
(40, 43, 84)	36	10.572	Optima
(40, 40, 75)	36	0.155	Optima
(40, 45, 84)	36	9.841	Optima
(40, 40, 76)	36	0.135	Optima
(40, 37, 75)	42	81.125	Optima
(41, 37, 75)	42	42.843	Factible
(41, 24, 113)	56	73.306	Mejor
(45, 35, 61)	52	164.774	Factible
(45, 30, 84)	48	6.114	Factible
(45, 32, 84)	46	73.444	Mejor
(50, 45, 109)	24	2.568	Factible
(50, 50, 91)	24	0.068	Factible
(55, 40, 80)	36	0.553	Optima
(60, 20, 141)	27	0.922	Factible
(62, 39, 92)	24	2.761	Factible
(64, 39, 93)	24	2.335	Factible
(69, 40, 95)	20	0.951	Factible

(125, 19, 118)	20	1.134	Optima
(130, 55, 75)	12	0.013	Optima
(130, 45, 62)	16	0.228	Factible
(135, 29, 47)	32	11.772	Factible
(135, 29, 46)	32	12.889	Factible
(138, 30, 47)	29	9.024	Factible
(140, 49, 52)	16	0.43	Optima
(160, 33, 46)	19	3.191	Factible
(164, 67, 73)	4	0.031	Optima
(170, 85, 45)	6	0.073	Factible
(171, 35, 70)	11	0.051	Factible
(185, 55, 55)	10	0.013	Optima
(185, 55, 56)	10	0.097	Optima
(190, 175, 45)	2	0.026	Optima
(195, 36, 130)	6	0.066	Optima

- Estrategia #6

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	72	1.832	Optima
(30, 30, 84)	72	0.527	Optima
(40, 35, 75)	56	5.953	Factible
(40, 43, 84)	34	21.819	Factible
(40, 40, 75)	36	0.228	Optima
(40, 45, 84)	34	21.084	Factible
(40, 40, 76)	36	0.222	Optima
(40, 37, 75)	36	34.969	Factible
(41, 37, 75)	36	43.084	Factible
(41, 24, 113)	48	36.057	Factible
(45, 35, 61)	60	102.66	Optima

(45, 30, 84)	46	6.845	Factible
(45, 32, 84)	46	33.028	Mejor
(50, 45, 109)	20	3.435	Factible
(50, 50, 91)	20	0.048	Factible
(55, 40, 80)	28	0.421	Factible
(60, 20, 141)	24	0.46	Factible
(62, 39, 92)	19	2.898	Factible
(64, 39, 93)	19	2.756	Factible
(69, 40, 95)	15	2.204	Factible
(125, 19, 118)	20	4.206	Optima
(130, 55, 75)	12	0.06	Optima
(130, 45, 62)	14	0.97	Factible
(135, 29, 47)	32	34.446	Factible
(135, 29, 46)	32	37.01	Factible
(138, 30, 47)	29	25.033	Factible
(140, 49, 52)	16	0.979	Optima
(160, 33, 46)	19	11.45	Factible
(164, 67, 73)	4	0.043	Optima
(170, 85, 45)	6	0.063	Factible
(171, 35, 70)	11	0.04	Factible
(185, 55, 55)	10	0.041	Optima
(185, 55, 56)	10	0.117	Optima
(190, 175, 45)	2	0.023	Optima
(195, 36, 130)	6	0.072	Optima

Para el contenedor 260x240x110:

- Estrategia #1:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	148	3.871	Factible
(30, 30, 84)	146	3.343	Factible

(40, 35, 75)	99	21.36	Factible
(40, 43, 84)	66	213.264	Factible
(40, 40, 75)	90	1.145	Optima
(40, 45, 84)	67	172.784	Factible
(40, 40, 76)	90	0.844	Optima
(40, 37, 75)	90	602.665	Mejor
(41, 37, 75)	90	777.988	Mejor
(41, 24, 113)	102	282.012	Mejor
(45, 35, 61)	106	949.58	Mejor
(45, 30, 84)	89	46.668	Factible
(45, 32, 84)	88	285.195	Factible
(50, 45, 109)	40	15.728	Optima
(50, 50, 91)	38	0.086	Factible
(55, 40, 80)	58	2.391	Factible
(60, 20, 141)	58	1.723	Factible
(62, 39, 92)	36	19.492	Factible
(64, 39, 93)	36	23.44	Factible
(69, 40, 95)	30	11.881	Factible
(125, 19, 118)	37	28.776	Factible
(130, 55, 75)	18	0.104	Factible
(130, 45, 62)	24	3.048	Factible
(135, 29, 47)	45	109.185	Factible
(135, 29, 46)	44	108.95	Factible
(138, 30, 47)	47	87.743	Factible
(140, 49, 52)	20	3.343	Factible
(160, 33, 46)	32	43.792	Factible
(164, 67, 73)	7	0.118	Optima
(170, 85, 45)	11	0.146	Factible
(171, 35, 70)	19	0.053	Optima
(185, 55, 55)	16	0.013	Factible
(185, 55, 56)	16	1.253	Factible

(190, 175, 45)	4	0.042	Optima
(195, 36, 130)	12	0.211	Factible

- Estrategia #2:

Caja	Total Cajas	Total Tiempo	Estado Respuesta
(30, 30, 85)	148	3.423	Factible
(30, 30, 84)	148	2.804	Factible
(40, 35, 75)	94	19.806	Factible
(40, 43, 84)	80	153.493	Mejor
(40, 40, 75)	89	0.678	Factible
(40, 45, 84)	74	142.088	Factible
(40, 40, 76)	89	0.719	Factible
(40, 37, 75)	90	501.415	Mejor
(41, 37, 75)	88	565.162	Mejor
(41, 24, 113)	97	235.75	Mejor
(45, 35, 61)	0	2643.089	Factible
(45, 30, 84)	98	50.265	Mejor
(45, 32, 84)	89	342.751	Factible
(50, 45, 109)	36	16.247	Factible
(50, 50, 91)	34	0.078	Factible
(55, 40, 80)	60	2.082	Factible
(60, 20, 141)	58	2.174	Factible
(62, 39, 92)	48	21.119	Factible
(64, 39, 93)	48	25.888	Factible
(69, 40, 95)	31	10.437	Factible
(125, 19, 118)	36	23.314	Factible
(130, 55, 75)	18	0.09	Factible
(130, 45, 62)	30	2.484	Factible
(135, 29, 47)	45	86.077	Factible
(135, 29, 46)	44	85.603	Factible

(138, 30, 47)	47	69.73	Factible
(140, 49, 52)	20	2.712	Factible
(160, 33, 46)	32	34.696	Factible
(164, 67, 73)	7	0.098	Optima
(170, 85, 45)	11	0.111	Factible
(171, 35, 70)	19	0.049	Optima
(185, 55, 55)	16	0.011	Factible
(185, 55, 56)	16	1.004	Factible
(190, 175, 45)	4	0.037	Optima
(195, 36, 130)	12	0.167	Factible

- Estrategia #3:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	140	4.849	Factible
(30, 30, 84)	140	4.125	Factible
(40, 35, 75)	99	29.944	Factible
(40, 43, 84)	80	270.97	Mejor
(40, 40, 75)	90	1.256	Optima
(40, 45, 84)	65	236.53	Factible
(40, 40, 76)	90	1.259	Optima
(40, 37, 75)	90	923.97	Mejor
(41, 37, 75)	90	1061.779	Mejor
(41, 24, 113)	96	239.934	Mejor
(45, 35, 61)	108	1526.685	Mejor
(45, 30, 84)	91	67.162	Factible
(45, 32, 84)	89	441.046	Factible
(50, 45, 109)	32	25.248	Factible
(50, 50, 91)	40	0.155	Optima
(55, 40, 80)	56	3.919	Factible
(60, 20, 141)	58	2.693	Factible
(62, 39, 92)	42	34.182	Factible

(64, 39, 93)	42	41.098	Factible
(69, 40, 95)	36	21.128	Factible
(125, 19, 118)	38	53.451	Factible
(130, 55, 75)	18	0.203	Factible
(130, 45, 62)	30	5.437	Factible
(135, 29, 47)	52	200.446	Mejor
(135, 29, 46)	52	288.454	Mejor
(138, 30, 47)	51	242.102	Mejor
(140, 49, 52)	20	9.991	Factible
(160, 33, 46)	32	119.992	Factible
(164, 67, 73)	7	0.317	Optima
(170, 85, 45)	11	0.383	Factible
(171, 35, 70)	19	0.143	Optima
(185, 55, 55)	16	0.033	Factible
(185, 55, 56)	16	3.774	Factible
(190, 175, 45)	4	0.125	Optima
(195, 36, 130)	12	0.599	Factible

- Estrategia #4:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	148	3.892	Factible
(30, 30, 84)	148	3.627	Factible
(40, 35, 75)	89	28.578	Factible
(40, 43, 84)	80	311.016	Mejor
(40, 40, 75)	89	2.03	Factible
(40, 45, 84)	75	316.183	Mejor
(40, 40, 76)	89	1.644	Factible
(40, 37, 75)	89	940.621	Mejor
(41, 37, 75)	88	1051.838	Mejor
(41, 24, 113)	104	202.193	Mejor

(45, 35, 61)	102	1447.22	Mejor
(45, 30, 84)	96	52.628	Mejor
(45, 32, 84)	90	355.473	Factible
(50, 45, 109)	36	17.005	Factible
(50, 50, 91)	32	0.105	Factible
(55, 40, 80)	60	2.765	Factible
(60, 20, 141)	56	1.801	Factible
(62, 39, 92)	48	23.934	Factible
(64, 39, 93)	48	29.148	Factible
(69, 40, 95)	33	13.63	Factible
(125, 19, 118)	30	36.754	Factible
(130, 55, 75)	18	0.14	Factible
(130, 45, 62)	30	4.012	Factible
(135, 29, 47)	42	133.536	Factible
(135, 29, 46)	42	139.57	Factible
(138, 30, 47)	44	109.278	Factible
(140, 49, 52)	20	4.18	Factible
(160, 33, 46)	43	54.49	Mejor
(164, 67, 73)	7	0.147	Optima
(170, 85, 45)	11	0.166	Factible
(171, 35, 70)	19	0.082	Optima
(185, 55, 55)	16	0.012	Factible
(185, 55, 56)	16	1.608	Factible
(190, 175, 45)	4	0.049	Optima
(195, 36, 130)	12	0.268	Factible

- Estrategia #5:

Caja	**Total Cajas**	**Total Tiempo**	**Estado Respuesta**
(30, 30, 85)	148	44.954	Factible
(30, 30, 84)	148	52.469	Factible

(40, 35, 75)	90	137.987	Factible
(40, 43, 84)	72	313.818	Factible
(40, 40, 75)	90	9.738	Optima
(40, 45, 84)	0	858.667	Factible
(40, 40, 76)	90	8.727	Optima
(40, 37, 75)	0	1920.405	Factible
(41, 37, 75)	0	2382.735	Factible
(41, 24, 113)	0	885.781	Factible
(45, 35, 61)	0	2749.26	Factible
(45, 30, 84)	97	156.129	Mejor
(45, 32, 84)	0	899.787	Factible
(50, 45, 109)	40	27.955	Optima
(50, 50, 91)	40	0.139	Optima
(55, 40, 80)	60	7.585	Factible
(60, 20, 141)	60	9.416	Optima
(62, 39, 92)	50	46.061	Optima
(64, 39, 93)	50	32.321	Optima
(69, 40, 95)	38	13.526	Factible
(125, 19, 118)	30	16.355	Factible
(130, 55, 75)	20	0.118	Factible
(130, 45, 62)	28	2.318	Factible
(135, 29, 47)	43	62.733	Factible
(135, 29, 46)	45	61.351	Factible
(138, 30, 47)	42	50.221	Factible
(140, 49, 52)	20	2.1	Factible
(160, 33, 46)	38	24.246	Factible
(164, 67, 73)	6	0.069	Factible
(170, 85, 45)	14	0.11	Factible
(171, 35, 70)	19	0.041	Optima
(185, 55, 55)	16	0.008	Factible
(185, 55, 56)	16	0.772	Factible

(190, 175, 45)	4	0.034	Optima
(195, 36, 130)	12	0.138	Factible

- Estrategia #6:

Caja	Total Cajas	Total Tiempo	Estado Respuesta
(30, 30, 85)	138	123.552	Factible
(30, 30, 84)	145	122.84	Factible
(40, 35, 75)	99	115.254	Factible
(40, 43, 84)	72	288.304	Factible
(40, 40, 75)	90	8.558	Optima
(40, 45, 84)	68	263.71	Factible
(40, 40, 76)	90	6.485	Optima
(40, 37, 75)	0	2165.442	Factible
(41, 37, 75)	0	2561.032	Factible
(41, 24, 113)	96	263.038	Mejor
(45, 35, 61)	0	3599.126	Factible
(45, 30, 84)	92	171.868	Factible
(45, 32, 84)	85	383.493	Factible
(50, 45, 109)	40	16.227	Optima
(50, 50, 91)	38	0.13	Factible
(55, 40, 80)	59	3.839	Factible
(60, 20, 141)	58	1.861	Factible
(62, 39, 92)	42	22.475	Factible
(64, 39, 93)	42	26.158	Factible
(69, 40, 95)	39	12.049	Factible
(125, 19, 118)	37	28.201	Factible
(130, 55, 75)	21	0.134	Factible
(130, 45, 62)	30	2.917	Factible
(135, 29, 47)	51	133.619	Mejor
(135, 29, 46)	56	132.11	Mejor

(138, 30, 47)	51	107.729	Mejor
(140, 49, 52)	20	3.426	Factible
(160, 33, 46)	33	50.077	Factible
(164, 67, 73)	7	0.114	Optima
(170, 85, 45)	11	0.139	Factible
(171, 35, 70)	18	0.073	Factible
(185, 55, 55)	15	0.026	Factible
(185, 55, 56)	16	1.309	Factible
(190, 175, 45)	4	0.04	Optima
(195, 36, 130)	12	0.243	Factible